

Preface

Deep learning (DL) is a quickly evolving field, which has demonstrated amazing results in performing tasks that traditionally have been performed well only by humans. Examples of such tasks are image classification, generating natural language descriptions of images, natural language translation, speech-to-text, and text-to-speech conversion.

Learning Deep Learning (this book, hereafter known as LDL) quickly brings you up to speed on the topic. It teaches how DL works, what it can do, and gives you some practical experience, with the overall objective of giving you a solid foundation for further learning.

In this book, we use green text boxes like this one to highlight concepts that we find extra important. The intent is to ensure that you do not miss key concepts. Let us begin by pointing out that we find **Deep Learning** important.

You will learn about the perceptron and other artificial neurons. They are the fundamental building blocks of deep neural networks that have enabled the DL revolution. You will learn about fully connected feedforward networks and convolutional networks. You will apply these networks to solve practical problems, such as predicting housing prices based on a large number of variables or identifying to which category an image belongs. Figure P-1 shows examples of such categories and images.

You will also learn about ways to represent words from a natural language using an encoding that captures some of the semantics of the encoded words. You will then use these encodings together with a recurrent neural network to create a neural-based natural language translator. This translator can automatically translate simple sentences from English to French or other similar languages, as illustrated in Figure P-2.

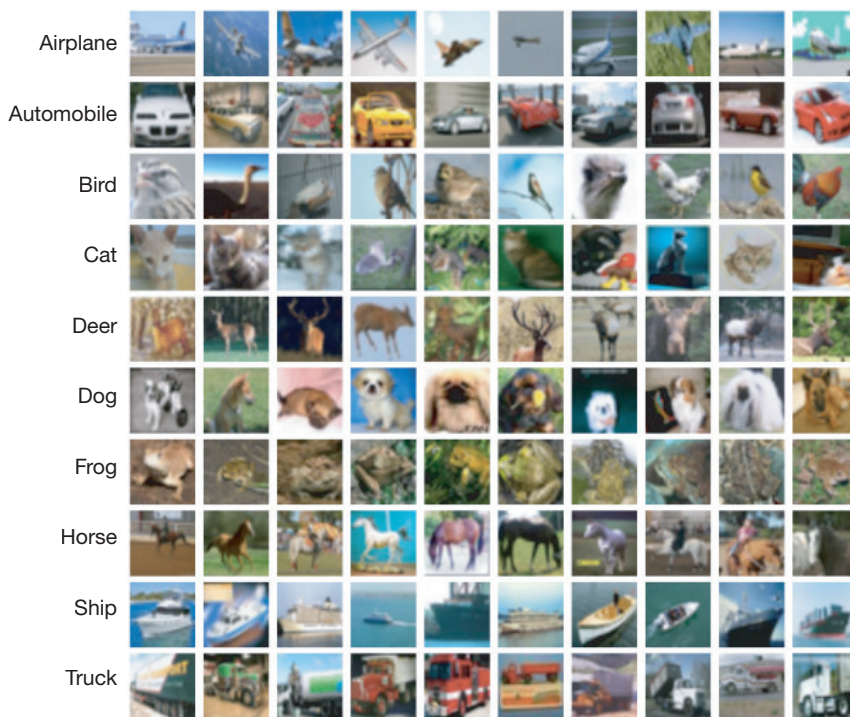


Figure P-1 Categories and example images from the CIFAR-10 dataset (Krizhevsky, 2009). This dataset will be studied in more detail in Chapter 7. (Image source: <https://www.cs.toronto.edu/~kriz/cifar.html>)

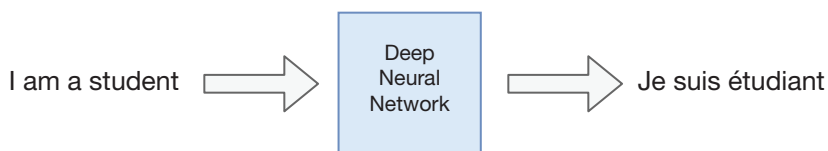


Figure P-2 A neural network translator that takes a sentence in English as input and produces the corresponding sentence in French as output

Finally, you will learn how to build an image-captioning network that combines image and language processing. This network takes an image as an input and automatically generates a natural language description of the image.

What we just described represents the main narrative of LDL. Throughout this journey, you will learn many other details. In addition, we end with a medley of additional important topics. We also provide appendixes that dive deeper into a collection of the discussed topics.

What Is Deep Learning?

We do not know of a crisp definition of what DL is, but one attempt is that *DL is a class of machine learning algorithms that use multiple layers of computational units where each layer learns its own representation of the input data. These representations are combined by later layers in a hierarchical fashion.* This definition is somewhat abstract, especially given that we have not yet described the concept of layers and computational units, but in the first few chapters, we provide many more concrete examples of what this means.

A fundamental part of DL is the deep neural network (DNN), a namesake of the biological neuron, by which it is loosely inspired. There is an ongoing debate about how closely the techniques within DL do mimic activity in a brain, where one camp argues that using the term *neural* network paints the picture that it is more advanced than it is. Along those lines, they recommend using the terms *unit* instead of *artificial neuron* and just *network* instead of *neural network*. No doubt, DL and the larger field of artificial intelligence (AI) have been significantly hyped in mainstream media. At the time of writing this book, it is easy to get the impression that we are close to creating machines that think like humans, although lately, articles that express some doubt are more common. After reading this book, you will have a more accurate view of what kind of problems DL can solve. In this book, we choose to freely use the words *neural network* and *neuron* but recognize that the algorithms presented are more tied to machine capabilities than to how an actual human brain works.

In this book, we use red text boxes like this one when we feel the urge to state something that is somewhat beside the point, a subjective opinion or of similar nature. You can safely ignore these boxes altogether if you do not find them adding any value to your reading experience.

Let us dive into this book by stating the opinion that it is a little bit of a buzz killer to take the stance that our cool DNNs are not similar to the brain. This is especially true for somebody picking up this book after reading about machines with superhuman abilities in the mainstream media. To keep the illusion alive, we sometimes allow ourselves to dream a little bit and make analogies that are not necessarily that well founded, but to avoid misleading you, we try not to dream outside of the red box.

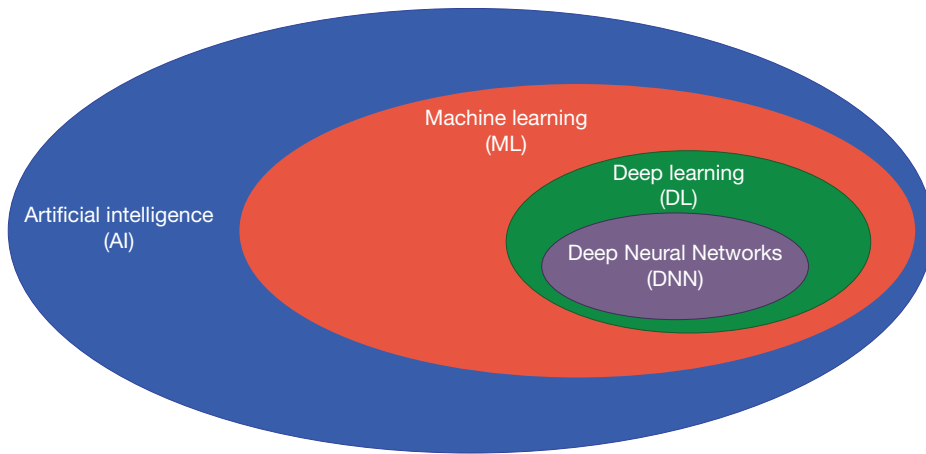


Figure P-3 Relationship between artificial intelligence, machine learning, deep learning, and deep neural networks. The sizes of the different ovals do not represent the relative size of one field compared to another.

To put DL and DNNs into context, Figure P-3 shows how they relate to the machine learning (ML) and AI fields. DNN is a subset of DL. DL in turn is a subset of the field of ML, which in turn is a subset of the greater field of AI.

Deep neural network (DNN) is a subset of DL.

DL is a subset of **machine learning (ML)**, which is a subset of **artificial intelligence (AI)**.

In this book, we choose not to focus too much on the exact definition of DL and its boundaries, nor do we go into the details of other areas of ML or AI. Instead, we focus on details of what DNNs are and the types of tasks to which they can be applied.

Brief History of Deep Neural Networks

In the last couple of sections, we loosely referred to networks without describing what a network is. The first few chapters in this book discuss network architectures in detail, but at this point, it is sufficient to think of a network as

an opaque system that has inputs and outputs. The usage model is to present something, for example, an image or a text sequence, as inputs to the network, and the network will produce something useful on its outputs, such as an interpretation of what the image contains, as in Figure P-4, or a natural language translation in a different language, as was shown in Figure P-2.

As previously mentioned, a central piece of a neural network is the artificial neuron. The first model of an artificial neuron was introduced in 1943 (McCulloch and Pitts, 1943), which started the first wave of neural network research. The McCulloch and Pitts neuron was followed in 1957 by the Rosenblatt perceptron (Rosenblatt, 1958). A key contribution from the perceptron was its associated automated learning algorithm that demonstrated how a system could learn desired behavior. Details of how the perceptron works are found in Chapter 1. The perceptron has some fundamental limitations, and although it was shown that these limitations can be overcome by combining multiple perceptrons into a multilayer network, the original learning algorithm did not extend to multilayer networks. According to a common narrative, this resulted in neural network research falling out of fashion. This is often referred to as the first AI winter, which was allegedly caused by a book by Minsky and Papert (1969). In this book, they raised the absence of a learning algorithm for multilayer networks as a serious concern.

We note that in the days of Rosenblatt's publications, they were certainly not shy about comparing their work with the human brain. In reading about the Rosenblatt perceptron (Rosenblatt, 1958), we see that the first paper he references is called "Design for a Brain."

This topic and narrative are controversial. Olazaran (1996) has studied whether the statements of Minsky and Papert had been misrepresented. Further, Schmidhuber (2015) pointed out that there did exist a learning algorithm for multilevel networks (Ivakhnenko and Lapa, 1965) four years before the book by Minsky and Papert was published.

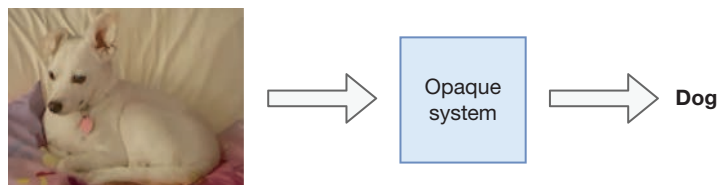


Figure P-4 A deep neural network as an opaque system that can take an image as an input and then output an indication of what type of object is in the image

The second wave of neural network research was initiated in the 1980s. It was heavily influenced by a paper that described the backpropagation algorithm for automatic training of multilayer networks (Rumelhart et al., 1986). Rumelhart and colleagues showed that this algorithm could be used to overcome the limitations of the perceptron. In the study, they explicitly pointed out that they believed this addressed the concerns raised by Minsky and Papert. Rumelhart and colleagues popularized the backpropagation algorithm in the context of neural networks, but it was not the first occurrence of the algorithm in the literature. The algorithm was applied to a similar problem domain in 1970 (Linnainmaa, 1970). Werbos (1981) described it in the context of neural networks in 1981.

Details of how this algorithm works are found in Chapter 3. An important outcome of this second wave of neural network research was the development of LeNet in 1989. It was a convolutional neural network (CNN), which was shown to be able to recognize handwritten zip codes (LeCun et al., 1990). It built on Fukushima's *Neocognitron* (Fukushima, 1980), which we believe is the first published CNN.

An enhanced version of LeNet was later used by major US banks to read handwritten checks, and it thereby became one of the first big commercial applications of neural networks. Convolutional neural networks are described in detail in Chapter 7. Despite the progress, neural networks fell out of fashion yet again, partly because the limited computational capability at the time prevented the networks from scaling to larger problems and partly because other traditional ML approaches were perceived as better alternatives.

The third wave of neural network research was enabled by a combination of algorithmic progress, availability of massive datasets, and the ability to use graphics processing units (GPU) for general purpose computing. From an outsider perspective, all this came together in 2012. At that point, the field had been rebranded as DL and was popularized in large part due to AlexNet (Krizhevsky et al., 2012), which was a CNN that scored significantly higher than any other participant in a computer vision competition known as the ImageNet challenge.

In reality, this third wave was enabled by persistent research groups who had continued to perform neural network research in the 1990s and first decade of the 2000s. These insiders started using the term *deep networks* in 2006. Further, the ImageNet challenge was not the first competition in which neural networks, some of which were GPU accelerated, beat more traditional techniques.

For example, Graves and colleagues (2009) won competitions in handwriting recognition with a neural network in 2009. Similarly, Ciresan and colleagues (2011) used a GPU accelerated network for image classification in 2011.

This work was shortly followed by similar breakthroughs in other fields, which have led to the DL boom that is still ongoing as of the writing of this book. The rest of this book will describe some of these key findings and how they can be applied in practice. For a more detailed description of the history of DL, we recommend Schmidhuber's (2015) overview.

Is This Book for You?

There are already many books on this topic, and different people like to approach subjects in different ways. In this book, we try to cut to the chase while still providing enough background to give you a warm fuzzy feeling that you understand why the techniques work. We decided to *not* start the book with an overall introduction to the field of traditional ML. Although we believe that anybody who wants to get serious about DL needs to also master traditional ML, we do not believe that it is necessary to first learn about traditional ML before learning the basics of DL. We even believe that having to first get through multiple chapters that do not directly discuss DL can be a barrier to entry for many people.

In this book, we use yellow text boxes like this one to highlight things that we otherwise do not discuss or explore in detail but nonetheless think are important for you to learn at some point. We believe that an important part of learning about a new topic is to not only acquire some basic skills but also get some insights into what the next steps are. We use the yellow boxes to signal to you that at this point it is perfectly fine to ignore a certain topic, but it will be important to learn as a next step.

Let us now begin by stating that it is important to know about **traditional ML** if you want to get serious about DL, but you can wait to learn about traditional ML until you have gotten a taste of DL.

Not starting the book with traditional ML techniques is an attempt to avoid one of the buzz killers that we have found in other books. One very logical, and therefore typical, way of introducing DL is to first describe what ML is and, as such, to start with a very simple ML technique, namely, linear regression. It is easy, as an excited beginner, to be a little disheartened when you expect to learn about cool techniques to classify cat images and instead get stuck reading a discussion about fitting a straight line to a set of random data points using mathematics that seem completely unrelated to DL. We instead try to take the quickest, while still logical, path to getting to image classification to provide you with some instant satisfaction, but you will notice that we still sneak in some references and comparisons to linear regression over time.

Apart from deciding whether to include traditional ML as a topic, any author of a book on DL needs to take a position on whether to include code examples and how deeply to dive into the mathematics. Our view is that because DL is an applied field, a book on this topic needs to contain a good mix of theory and practice, so code examples are necessary. We also believe that many topics in DL are inherently mathematical, and it is necessary to include some of the mathematics to provide a good description of how things work. With that background, we try to describe certain concepts from different angles using a good mix of elements:

- Figures
- Natural language (English) descriptions
- Programming code snippets
- Mathematical formulas

Readers who master all of the preceding might find some descriptions redundant, but we believe that this is the best way of making the book accessible to a large audience.

This book does not aim to include details about all the most recent and advanced techniques in the DL field. Instead, we include concepts and techniques that we believe are fundamental to understanding the latest developments in the field. Some of the appendixes describe how some major architectures are built on these concepts, but most likely, even better architectures will emerge. Our goal is to give you enough knowledge to enable you to continue learning by reading more recent research papers. Therefore, we have also decided to sprinkle references throughout the book to enable you to follow up on topics that you find extra

interesting. However, it has been our intention to make the book self-contained so that you should never need to look up a reference to be able to follow the explanations in the book. In some cases, we include references to things that we do not explain but mention only in passing. In those cases, we try to make it clear that it is meant as future reading instead of being a central element of the book.

The references in the book are strictly for future reading and should not be necessary to read to be able to understand the main topics of the book.

Is DL Dangerous?

There are plenty of science fiction books and movies that depict AI as a threat against humanity. Machines develop a form of consciousness and perceive humans as a threat and therefore decide to destroy us. There have also been thought experiments about how an AI accidentally destroys the human species as a side effect of trying to deliver on what it is programmed to do. One example is the paperclip maximizer (Bostrom, 2003), which is programmed with the goal of making as many paper clips as possible. In order to do so, it might kill all human beings to free up atoms needed to make paper clips. The risk that these exact scenarios will play out in practice is probably low, but researchers still see future powerful AIs as a significant risk.

More urgently, DL has already been shown to come with serious unintended consequences and malignant use. One example is a study of a commercially available facial recognition system (Buolamwini and Gebru, 2018) used by law enforcement. Although the system achieved 99% accuracy on lighter-skinned men, its accuracy on darker-skinned women was only 65%, thereby putting them at much greater risk of being incorrectly identified and possibly wrongly accused of crimes. An example of malignant use of DL is fake pornography (Dickson, 2019) whereby the technology is used to make it appear as if a person (often a celebrity) is featured in a pornographic video.

DL learns from data created by humans and consequently runs the risk of learning and even amplifying human biases. This underscores the need for taking a responsible approach to DL and AI. Historically, this topic has largely been neglected, but more recently started to receive more attention. A powerful demonstration can be found on the website of the Algorithmic Justice League (Buolamwini, n.d.) with a video showing how a face detection system fails to detect the face of a dark-skinned woman (Buolamwini) until she puts on a white mask.

Another example is the emergence of algorithmic auditing, where researchers identify and report human biases and other observed problems in commercial systems (Raji and Buolamwini, 2019). Researchers have proposed to document known biases and intended use cases of any released system to mitigate these problems. This applies both to the data used to create such systems (Geburu, et al., 2018) and to the released DL model itself (Mitchell et al., 2018). Thomas suggests a checklist of questions to guide DL practitioners throughout the course of a project to avoid ethical problems (Thomas, 2019). We touch on these topics throughout the book. We also provide resources for further reading in Chapter 18.

Choosing a DL Framework

As a practitioner of DL, you will need to decide what DL framework to use. A DL framework provides functionality that handles much of the low-level details when implementing DL models. Just as the DL field is rapidly evolving, so are the different frameworks. To mention a few, Caffe, Theano, MXNet, Torch, TensorFlow, and PyTorch have all been influential throughout the current DL boom. In addition to these full-fledged frameworks, there are specialized frameworks such as Keras and TensorRT. Keras is a high-level API that makes it easier to program for some of these frameworks. TensorRT is an inference optimizer and runtime engine that can be used to run models built and trained by many of the mentioned frameworks.

As of the writing of this book, our impression is that the two most popular full-fledged frameworks are TensorFlow and PyTorch, where TensorFlow nowadays includes native support for the Keras API. Another significant framework is MXNet. Models developed in either of these frameworks can be deployed using the TensorRT inference engine.

Deciding on what DL framework to use can be viewed as a life-changing decision. Some people would say that it is comparable to choosing a text editor or a spouse. We do not share that belief but think that the world is big enough for multiple competing solutions. We decided to provide programming examples in both TensorFlow and PyTorch for this book. The TensorFlow examples are printed in the book itself, but equivalent examples in PyTorch, including detailed descriptions, can be found on the book's website. We suggest that you pick a framework that you like or one that makes it easy to collaborate with people you interact with.

The programming examples in this book are provided in a TensorFlow version using the Keras API (printed in the book) as well as in a PyTorch version (online). Appendix I contains information about how to install TensorFlow and PyTorch, as well as a description of some of the key differences between the two frameworks.

Prerequisites for Learning DL

DL combines techniques from a number of different fields. If you want to get serious about DL, and particularly if you want to do research and publish your findings, over time you will need to acquire advanced knowledge within the scope of many of these skillsets. However, we believe that it is possible to get started with DL with little or partial knowledge in these areas. The sections that follow list the areas we find important, and in each section, we list the minimum set of knowledge that we think you need in order to follow this book.

STATISTICS AND PROBABILITY THEORY

Many DL problems do not have exact answers, so a central theme is probability theory. As an example, if we want to classify objects in an image, there is often uncertainty involved, such as how certain our model is that an object of a specific category, such as a cat, is present in the picture. Further, we might want to classify the type of cat—for example, is it a tiger, lion, jaguar, leopard, or snow leopard? The answer might be that the model is 90% sure that it is a jaguar, but there is a 5% probability that it is a leopard and so on. This book does not require deep knowledge in statistics and probability theory. We do expect you to be able to compute an arithmetic mean and understand the basic concept of probability. It is helpful, although not strictly required, if you know about variance and how to standardize a random variable.

LINEAR ALGEBRA

As you will learn in Chapter 1, the fundamental building block in DL is based on calculating a weighted sum of variables, which implies doing many additions and multiplications. Linear algebra is a field of mathematics that enables us to

describe such calculations in a compact manner. This book frequently specifies formulas containing vectors and matrices. Further, calculations involve

- Dot products
- Matrix-vector multiplications
- Matrix-matrix multiplications

If you have not seen these concepts in the past, you will need to learn about them to follow the book. However, Chapter 1 contains a section that goes through these concepts. We suggest that you read that first and then assess whether you need to pick up a book about linear algebra.

CALCULUS

As you will learn in Chapters 2 and 3, the learning part in DL is based on minimizing the value of a function known as a *loss function* or *error function*. The technique used to minimize the loss function builds on the following concepts from calculus:

- Computing the derivative of a function of a single variable
- Computing partial derivatives of a function of multiple variables
- Calculating derivatives using the chain rule of calculus

However, just as we do for linear algebra, we provide sections that go through the basics of these concepts. These sections are found in Chapters 2 and 3.

NUMERICAL METHODS FOR CONSTRAINED AND UNCONSTRAINED OPTIMIZATION

In DL, it is typically not feasible to find an analytical solution when trying to minimize the loss function. Instead, we rely on numerical optimization methods. The most prevalent method is an iterative method known as *gradient descent*. It is helpful if you already know something about iterative methods and finding extreme points in continuous functions. However, we do not require prior knowledge of gradient descent, and we describe how it works before using it in Chapter 3.

PYTHON PROGRAMMING

It is hard to do anything except specific DL applications without some knowledge about programming in general. Further, given that the most popular DL frameworks are based on Python, it is highly recommended to acquire at least basic Python skills to enable trying out and modifying code examples. There are many good books on the topic of programming, and if you have basic programming skills, it should be relatively simple to get started with Python by just following tutorials at python.org. It is possible for nonprogrammers to read this book and just skip the coding sections, but if you intend to apply your DL skills in practice, you should learn the basics of Python programming.

You do not need to learn everything about Python to get started with DL. Many DL applications use only a small subset of the Python language, extended with heavy use of domain-specific DL frameworks and libraries. In particular, many introductory examples make little or no use of object-oriented programming constructs. A specific module that is used frequently is the *NumPy* (numerical Python) module that, among other things, provides data types for vectors and matrices. It is also common to use *pandas* (Python Data Manipulation Library) to manipulate multidimensional data, but we do not make use of *pandas* in this book.

The following Python constructs are frequent in most of the code examples in the book:

- Integer and floating point datatypes
- Lists and dictionaries
- Importing and using external packages
- NumPy arrays
- NumPy functions
- If-statements, for-loops, and while-loops
- Defining and calling functions
- Printing strings and numerical datatypes

- Plotting data with matplotlib
- Reading from and writing to files

In addition, many of the programming examples rely on constructs provided by a DL framework (TensorFlow in the book and PyTorch provided online). There is no need to know about these frameworks up front. The functionality is gradually introduced in the descriptions of the code examples. The code examples become progressively harder throughout the book, so if you are a beginner to coding, you will need to be prepared to spend some time honing your coding skills in parallel with reading the book.

DATA REPRESENTATION

Much of the DL mechanics are handled by highly optimized ML frameworks. However, your input data first needs to be converted into suitable formats that can be consumed by these frameworks. As such, you need to know something about the format of the data that you will use and, when applicable, how to convert it into a more suitable format. For example, for images, it is helpful to know the basics about RGB (red, green, blue) representation. Similarly, for the cases that use text as input data, it is helpful to know something about how characters are represented by a computer. In general, it is good to have some insight into how raw input data is often of low quality and needs to be cleaned. You will often find missing or duplicated data entries, timestamps from different time zones, and typos originating from manual processing. For the examples in this book, this is typically not a problem, but it is something you need to be aware of in a production setting.

About the Code Examples

You will find much overlap between the code examples in this book and code examples found in online tutorials as well as in other DL books (e.g., Chollet 2018; Glassner, 2018). Many of these examples have evolved from various published research papers in combination with publicly available datasets. (Datasets are described in more detail in Chapter 4.) In other words, we want to stress that we have not made up these examples from scratch, but they are heavily inspired by previously published work. However, we have done the actual implementation of these examples, and we have put our own touch on them to follow the organization of this book.

The longer code examples are broken up into smaller pieces and presented step by step interspersed throughout the text in the book. You should be able to just copy/paste or type each code snippet into a Python interpreter, but it is probably better to just put all code snippets for a specific code example in a single file and execute in a noninteractive manner. The code examples are also available for download both as regular Python files and as Jupyter notebooks at <https://github.com/NVDLI/LDL/>. See Appendix I for more details.

We were tempted to not provide downloadable versions of the code examples but instead force you to type them in yourself. After all, that is what we had to do in the 1980s when typing in a code listing from a computer magazine was a perfectly reasonable way of obtaining a new game. The youth of today with their app stores simply do not know how lucky they are.

In most chapters, we first present a basic version of a code example, and then we present results for variations of the program. We do not provide the full listings for all variations, but we try to provide all the necessary constructs in the book to enable you to do these variations yourself.

Modifying the code is left as an exercise for the reader. Hah, we finally got to say that!

Seriously, we do believe that modifying existing code is a good way of getting your hands dirty. However, there is no need to exactly recreate the variations we did. If you are new to programming, you can start with just tweaking existing parameter values instead of adding new code. If you already have more advanced coding skills, you can consider defining your own experiments based on what you find extra interesting.

DL algorithms are based on stochastic optimization techniques. As such, the results from an experiment may vary from time to time. That is, when you run a code example, you should not expect to get exactly the same result that is shown in the book. However, the overall behavior should be the same.

Another thing to note is that the chosen format, where we intersperse code throughout the book and explain each snippet, results in certain restrictions, such as minimizing the length of each program, and we have also tried to maintain

a linear flow and to not heavily modularize the code into classes and functions in most cases. Thus, instead of using sound coding practices to make the code examples easy to extend and maintain, focus is on keeping the examples small and readable.

That is a lame excuse for writing ugly code, but whatever works. . .

Another thing to consider is what kind of development environment is needed to follow this book. In our opinion, anybody who wants to do serious work in DL will need to get access to a hardware platform that provides specific acceleration for DL—for example, a suitable graphics processing unit (GPU). However, if you do not have access to a GPU-based platform just yet, the code examples in the first few chapters are small enough to be run on a somewhat modern central processing unit (CPU) without too much pain. That is, you can start with a vanilla setup using the CPU for the first few chapters and then spend the resources needed to get access to a GPU-accelerated platform¹ when you are getting to Chapter 7.

Medium term, you should get access to a GPU accelerated platform, but you can live with a standard CPU for the beginning of the book.

Instructions on how to set up a machine with the necessary development environment can be found in Appendix I, which also contains links to the code examples and datasets used in this book.

How to Read This Book

This book is written in a linear fashion and is meant to be read from beginning to end. We introduce new concepts in each chapter and frequently build on and refer back to discussions in previous chapters. It is often the case that we try to avoid introducing too many new concepts at once. This sometimes results in logically similar concepts being introduced in different chapters. However, we do sometimes take a step back and try to summarize a group of related techniques once they have all been introduced. You will see this for hidden units in Chapter 5,

1. Nothing prevents you from running all programming examples on a CPU, but in some cases, you might need to do it overnight.

output units in Chapter 6, and techniques to address vanishing and exploding gradients in Chapter 10.

Readers who are complete beginners to neural networks and DL (the core target audience of the book) will likely find the first four chapters more challenging to get through than the remainder of the book. We introduce many new concepts. There is a fair amount of mathematical content, and we implement a neural network from scratch in Python. We encourage you to still try to get through these four chapters, but we also think it is perfectly fine to skim through some of the mathematical equations if you find them challenging. In Chapter 5, we move on to using a DL framework, and you will find that it will handle many of the details under the hood, and you can almost forget about them.

APPENDIXES

This book ends with a number of appendixes. Appendixes A through D could have been included as regular chapters in the book. However, we wanted to avoid information overload for first-time readers. Therefore, we decided to put some of the material in appendixes instead because we simply do not think that you need to learn those concepts in order to follow the narrative of the book. Our recommendation if you are a complete beginner to ML and DL is to read these appendixes last.

If you feel that you already know the basics about ML or DL, then it can make sense for you to read the first four appendixes interspersed among other chapters during your first pass through the book. Appendix A can be read after Chapter 3. Appendix B logically follows Chapter 8. Appendix C naturally falls after Chapter 13. Finally, Appendix D extends topics presented in Chapter 15.

Alternatively, even if you are a beginner but want to learn more details about a specific topic, then do go ahead and read the appendix that relates to that topic in the order just presented.

Appendixes E through H are shorter and focus on providing background or additional detail on some very specific topics. Appendix I describes how to set up a development environment and how to access the programming examples. Appendix J contains cheat sheets that summarize many of the concepts described throughout the book.²

². Larger versions of these cheat sheets can be downloaded from <http://informit.com/title/9780137470358>.

GUIDANCE FOR READERS WHO DO NOT WANT TO READ ALL OF THIS BOOK

We recognize that some readers want to read this book in a more selective manner. This can be the case if you feel that you already have some of the basic skills or if you just want to learn about a specific topic. In this section, we provide some pointers for such readers, but this also means that we use some terminology that has not yet been introduced. If you are not interested in cherry picking chapters to read, then feel free to skip this section.

Figure P-5 illustrates three different envisioned tracks to follow depending on your interests. The leftmost track is what we just described, namely, to read the book from beginning to end.

If you are very interested in working with images and computer vision, we suggest that you read Appendix B about object detection, semantic segmentation, and instance segmentation. Further, the last few chapters of the book focus on natural language processing, and if that does not interest you, then we suggest that you skip Chapters 12 through 17. You should still skim Chapters 9 through 11 about recurrent neural networks. This track is shown in the middle of the figure.

If you want to focus mostly on language processing, then you can select the rightmost track. We suggest that you just skim Chapter 8 but do pay attention to the description of skip connections because it is referenced in later chapters. Then read Chapters 9 through 13, followed by Appendix C, then Chapters 14 and 15, and conclude with Appendix D. These appendixes contain additional content about word embeddings and describe GPT and BERT, which are important network architectures for language processing tasks.

Overview of Each Chapter and Appendix

This section contains a brief overview of each chapter. It can safely be skipped if you just want to cut to the chase and get started with LDL!

CHAPTER 1 – THE ROSENBLATT PERCEPTRON

The perceptron, a fundamental building block of a neural network, is introduced. You will learn limitations of the perceptron, and we show how to overcome

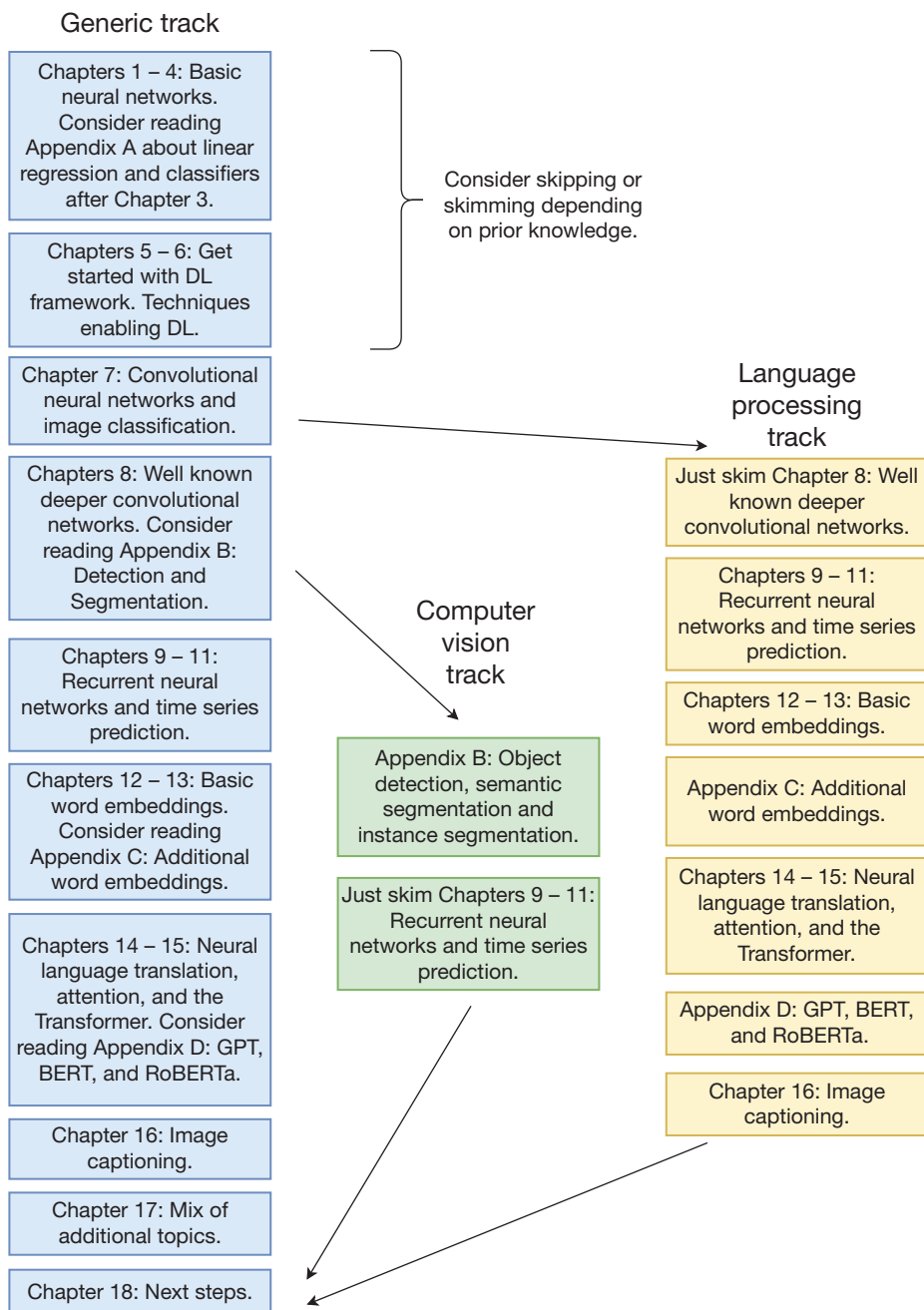


Figure P-5 Three different tracks to follow when reading this book

these limitations by combining multiple perceptrons into a network. The chapter contains some programming examples of how to implement a perceptron and its learning algorithm.

CHAPTER 2 – GRADIENT-BASED LEARNING

We describe an optimization algorithm known as *gradient descent* and the theory behind the perceptron learning algorithm. This is used as a stepping-stone in the subsequent chapter that describes the learning algorithm for multilevel networks.

CHAPTER 3 – SIGMOID NEURONS AND BACKPROPAGATION

We introduce the backpropagation algorithm that is used for automatic learning in DNNs. This is both described in mathematical terms and implemented as a programming example used to do binary classification.

CHAPTER 4 – FULLY CONNECTED NETWORKS APPLIED TO MULTICLASS CLASSIFICATION

This chapter describes the concept of datasets and how they can be divided into a training set and a test set. It also touches on a network's ability to generalize. We extend the neural network architecture to handle multiclass classification, and the programming example then applies this to the task of classifying handwritten digits. This programming example is heavily inspired by an example created by Nielsen (2015).

CHAPTER 5 – TOWARD DL: FRAMEWORKS AND NETWORK TWEAKS

The example from the previous chapter is reimplemented using a DL framework. We show how this framework vastly simplifies the code and enables us to model many variations on our network. Chapter 5 also introduces many techniques that are needed to enable training of deeper networks.

CHAPTER 6 – FULLY CONNECTED NETWORKS APPLIED TO REGRESSION

In this chapter, we study how a network can be used to predict a numerical value instead of classification problems studied in previous chapters. We do this with a programming example in which we apply the network to a regression problem

where we are trying to predict sales prices of houses based on a number of variables.

CHAPTER 7 – CONVOLUTIONAL NEURAL NETWORKS APPLIED TO IMAGE CLASSIFICATION

You will learn about the one type of network that initiated the DL boom in 2012, namely, the convolutional neural network, or just convolutional network. A CNN can be used in multiple problem domains, but it has been shown to be especially effective when applied to image classification/analysis. We explain how it works and walk through a programming example that uses a CNN to classify a more complex image dataset. In this example, instead of just distinguishing between different handwritten digits, we identify more complex object classes such as airplanes, automobiles, birds, and cats.

CHAPTER 8 – DEEPER CNNs AND PRETRAINED MODELS

Here we describe deeper CNNs such as GoogLeNet, VGG, and ResNet. As a programming example, we show how to download a pretrained ResNet implementation and how to use it to classify your own images.

CHAPTER 9 – PREDICTING TIME SEQUENCES WITH RECURRENT NEURAL NETWORKS

One limitation of the networks described in the previous chapters is that they are not well suited to handle data of different input lengths. Important problem domains such as text and speech often consist of sequences of varying lengths. This chapter introduces the recurrent neural network (RNN) architecture, which is well suited to handle such tasks. We use a programming example to explore how this network architecture can be used to predict the next data point in a time series.

CHAPTER 10 – LONG SHORT-TERM MEMORY

We discuss problems that prevent RNNs from learning long-term dependencies. We describe the long short-term memory (LSTM) technique that enables better handling of long sequences.

CHAPTER 11 – TEXT AUTOCOMPLETION WITH LSTM AND BEAM SEARCH

In this chapter, we explore how to use LSTM-based RNNs for longer-term prediction and introduce a concept known as *beam search*. We illustrate it with a programming example in which we build a network that can be used for autocompletion of text. This is a simple example of natural language generation (NLG), which is a subset of the greater field of natural language processing (NLP).

CHAPTER 12 – NEURAL LANGUAGE MODELS AND WORD EMBEDDINGS

The example in the previous chapter is based on individual characters instead of words. In many cases, it is more powerful to work with words and their semantics instead of working with individual characters. Chapter 12 introduces the concepts language models and word encodings in a vector space (also known as *embedding space*) that can be used to capture some important relationships between words. As code examples, we extend our autocompletion example to work with words instead of characters and explore how to create word vectors in an embedding space. We also discuss how to build a model that can do sentiment analysis on text. This is an example of natural language understanding (NLU), which is yet another subfield of NLP.

CHAPTER 13 – WORD EMBEDDINGS FROM word2vec AND GloVe

In this chapter, we discuss two popular techniques for creating word embeddings. We download a set of existing embeddings and show how they capture various semantic relationships between words.

CHAPTER 14 – SEQUENCE-TO-SEQUENCE NETWORKS AND NATURAL LANGUAGE TRANSLATION

At this point, we introduce a network known as a sequence-to-sequence network, which is a combination of two recurrent neural networks. A key property of such a network is that its output sequence can be of a different length than the input sequence. We combine this type of network with the word encodings studied in the previous chapter. We build a natural language translator that takes a word sequence in one language (e.g., French) as an input and outputs a word sequence in a different language (e.g., English). Further, the output might be a different number of words and in a different word order than the input word sequence. The

sequence-to-sequence model is an example of an architecture known as *encoder-decoder architecture*.

CHAPTER 15 – ATTENTION AND THE TRANSFORMER

In this chapter, we describe a technique known as *attention*, which can improve the accuracy of encoder-decoder architectures. We describe how it can be used to improve the neural machine translator from the previous chapter. We also describe the attention-based Transformer architecture. It is a key building block in many NLP applications.

CHAPTER 16 – ONE-TO-MANY NETWORK FOR IMAGE CAPTIONING

We describe in this chapter how a one-to-many network can be used to create textual descriptions of images and how to extend such a network with attention. A programming example implements this image-captioning network and demonstrates how it can be used to generate textual descriptions of a set of pictures.

CHAPTER 17 – MEDLEY OF ADDITIONAL TOPICS

Up until this point, we have organized topics so that they build on each other. In this chapter, we introduce a handful of topics that we did not find a good way of including in the previous chapters. Examples of such topics are autoencoders, multimodal learning, multitask learning, and neural architecture search.

CHAPTER 18 – SUMMARY AND NEXT STEPS

In the final chapter, we organize and summarize the topics discussed in earlier chapters to give you a chance to confirm that you have captured the key concepts described in the book. In addition to the summary, we provide some guidance to future reading tailored according to the direction you want to take—for example, highly theoretical versus more practical. We also discuss the topics of ethical AI and data ethics.

APPENDIX A – LINEAR REGRESSION AND LINEAR CLASSIFIERS

The focus of this book is DL. Our approach to the topic is to jump straight into DL without first describing traditional ML techniques. However, this appendix

does describe very basic ML topics so you can get an idea of how some of the presented DL concepts relate to more traditional ML techniques. This appendix logically follows Chapter 3.

APPENDIX B – OBJECT DETECTION AND SEGMENTATION

In this appendix, we describe techniques to detect and classify multiple objects in a single image. It includes both coarse-grained techniques that draw bounding boxes around the objects and fine-grained techniques that pinpoint the individual pixels in an image that correspond to a certain object. This appendix logically follows Chapter 8.

APPENDIX C – WORD EMBEDDINGS BEYOND word2vec AND GloVe

In this appendix, we describe some more elaborate techniques for word embeddings. In particular, these techniques can handle words that did not exist in the training dataset. Further, we describe a technique that can handle cases in which a word has a different meaning depending on its context. This appendix logically follows Chapter 13.

APPENDIX D – GPT, BERT, AND RoBERTa

This appendix describes architectures that build on the Transformer. These network architectures have resulted in significant improvements in many NLP tasks. This appendix logically follows Chapter 15.

APPENDIX E – NEWTON-RAPHSON VERSUS GRADIENT DESCENT

In Chapter 2, we introduce a mathematical concept technique known as *gradient descent*. This appendix describes a different method, known as *Newton-Raphson*, and how it relates to gradient descent.

APPENDIX F – MATRIX IMPLEMENTATION OF DIGIT CLASSIFICATION NETWORK

In Chapter 4, we include a programming example implementing a neural network in Python code. This appendix describes two different optimized variations of that programming example.

APPENDIX G – RELATING CONVOLUTIONAL LAYERS TO MATHEMATICAL CONVOLUTION

In Chapter 7, we describe convolutional neural networks. They are based on, and named after, a mathematical operation known as *convolution*. This appendix describes this connection in more detail.

APPENDIX H – GATED RECURRENT UNITS

In Chapter 10, we describe a network unit known as *long short-term memory* (LSTM). In this appendix, we describe a simplified version of this unit known as *gated recurrent unit* (GRU).

APPENDIX I – SETTING UP A DEVELOPMENT ENVIRONMENT

This appendix contains information about how to set up a development environment. This includes how to install a deep learning framework and where to find the code examples. It also contains a brief section about key differences between TensorFlow and PyTorch, which are the two DL frameworks used for the code examples in this book.

APPENDIX J – CHEAT SHEETS

This appendix contains a set of cheat sheets that summarize much of the content in this book. They are also available for download in a different form factor: <http://informit.com/title/9780137470358>.

Register your copy of *Learning Deep Learning* on the InformIT site for convenient access to updates and/or corrections as they become available. To start the registration process, go to informit.com/register and log in or create an account. Enter the product ISBN (9780137470358) and click Submit. Look on the Registered Products tab for an Access Bonus Content link next to this product, and follow that link to access any available bonus materials. If you would like to be notified of exclusive offers on new editions and updates, please check the box to receive email from us.

This page intentionally left blank

Acknowledgments

I am incredibly grateful for all help I have received in the process of writing this book. I would like to extend my warmest gratitude to all of you:

- Eric Haines for reading this book front to back and providing guidance and feedback throughout the whole process. Having had you as a sounding board and discussion partner was invaluable.
- Ankit Patel and Amanda Lam for believing in me. Without any prior interaction with me, you worked extra hours to figure out how to make this book happen. Thank you for finding the perfect publisher, and thanks to Jenny Chen for working out the agreement. Being represented by a professional team allowed me to focus solely on the content of the book.
- Nick Cohron, Orazio Gallo, Boris Ginsburg, Samuli Laine, Ryan Prenger, Raul Puri, Kevin Shih, and Sophie Tabac for providing expert feedback on the material. All of your comments greatly improved the book.
- Aaron Beddes and Torbjörn Ekman for reading an early manuscript and providing valuable feedback, which gave me confidence to engage all the people above.
- Anders Landin, Feihui Li, Niklas Lindström, Jatin Mitra, Clint Olsen, Sebastian Sylvan, and Johan Överby for pointing out various issues, both in the manuscript and in the code examples.
- Andy Cook for your vision of how to tie the book to efforts within the NVIDIA Deep Learning Institute, as well as your involvement in the cover art proposal, together with Sandra Froehlich and Chris Strach. Sandra and Chris also contributed with other aspects of style and branding. The original image of the R.Eva Robot was designed by Gavriil Klimov and Gregor Kopka.
- Anima Anandkumar and Craig Clawson for writing the forewords.

- All people who Pearson involved in the publishing process, in particular Debra Williams Cauley, Carol Lallier, Julie Nahil, Chuti Prasertsith, and Chris Zahn.
- Darrell Boggs for providing your support when I first introduced the idea of starting this project. Further, the following NVIDIA colleagues all played a role in making this happen by supporting the project or connecting me with the right people: Tomas Akenine-Möller, Anima Anandkumar, Jonathan Cohen, Greg Estes, Sanja Fidler, David Hass, Brian Kelleher, Will Ramey, and Mohammad Shoeybi.
- The research community and other authors. This book does not contain original ideas. Its focus is on describing published work from multiple sources in a common framework. This would not have been possible without the original publications as well as multiple books on the topic. I have done my best to list all these sources in the bibliography.

Finally, I am grateful to my wife, Jennifer, and my children, Sebastian and Sofia, for being understanding and enabling me to spend the time required to write this book. I also want to give credit to our dog, Babette, and late cat, Stella, because I used their pictures in various object classification examples.

About the Author

Magnus Ekman, PhD, is a Director of Architecture at NVIDIA Corporation. His doctorate is in computer engineering, and he holds multiple patents. He was first exposed to artificial neural networks in the late 1990s in his native country, Sweden. After some dabbling in evolutionary computation, he focused on computer architecture and relocated to Silicon Valley, where he lives with his wife, Jennifer, children, Sebastian and Sofia, and dog, Babette. He has previously worked with processor design and R&D at Sun Microsystems and Samsung Research America and has been involved in starting two companies, one of which (Skout) was later acquired by The Meet Group, Inc. In his current role at NVIDIA, he leads an engineering team working on CPU performance and power efficiency for chips targeting markets ranging from autonomous vehicles to data centers for artificial intelligence (AI).

As the deep learning (DL) field exploded in the past few years, fueled by NVIDIA's GPU technology and CUDA, Dr. Ekman found himself in the midst of a company expanding beyond computer graphics and becoming a DL powerhouse. As a part of that journey, he challenged himself to stay up to date with the most recent developments in the field. He considers himself an educator, and in the process of writing *Learning Deep Learning* (LDL) he partnered with the NVIDIA Deep Learning Institute (DLI), which offers hands-on training in AI, accelerated computing, and accelerated data science. He is thrilled about DLI's plans to add LDL to its existing portfolio of self-paced online courses; live, instructor-led workshops; educator programs; and teaching kits.

This page intentionally left blank

Chapter 1

The Rosenblatt Perceptron

This chapter describes the Rosenblatt perceptron and shows how it can be used. Chapters 3 and 5 describe how the perceptron has been modified over time to enable more advanced networks. The perceptron is an artificial neuron, that is, a model of a biological neuron. Therefore, it makes sense to first briefly describe the parts of a biological neuron, as shown in Figure 1-1.

A biological neuron consists of one cell body, multiple dendrites, and a single axon. The connections between neurons are known as synapses. The neuron receives stimuli on the dendrites, and in cases of sufficient stimuli, the neuron fires (also known as getting activated or excited) and outputs stimulus on its

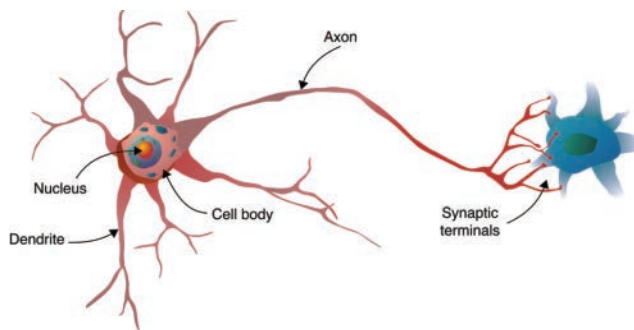


Figure 1-1 A biological neuron (Source: Glassner, A., Deep Learning: From Basics to Practice, The Imaginary Institute, 2018.)

axon, which is transmitted to other neurons that have synaptic connections to the excited neuron. Synaptic signals can be excitatory or inhibitory; that is, some signals can prevent a neuron from firing instead of causing it to fire.

The perceptron consists of a computational unit, a number of inputs (one of which is a special *bias input*, which is detailed later in this chapter), each with an associated input weight and a single output. The perceptron is shown in Figure 1-2.

The inputs are typically named $x_0, x_1, \dots, x_n$ in the case of n general inputs (x_0 being the bias input), and the output is typically named y . The inputs and output loosely correspond to the dendrites and the axon. Each input has an associated weight (w_i , where $i = 0, \dots, n$), which historically has been referred to as *synaptic weight* because it in some sense represents how strong the connection is from one neuron to another, but nowadays it is typically just called *weight* or *input weight*. For the perceptron, the output can take on only one of two values, -1 or 1 , but this constraint will be relaxed to a range of real values for other types of artificial neurons discussed in later chapters. The bias input is always 1. Each input value is multiplied by its corresponding weight before it is presented to the computational unit (the dashed rectangle with rounded corners in Fig. 1-2), which loosely corresponds to the cell body of a biological neuron.¹ The computational unit computes the sum of the weighted inputs and then applies a so-called activation function, $y = f(z)$, where z is the sum of the weighted inputs. The activation function for a perceptron is the sign function, also known as the signum function,² which evaluates to 1 if the input is 0 or higher and -1 otherwise. The sign function is shown in Figure 1-3.

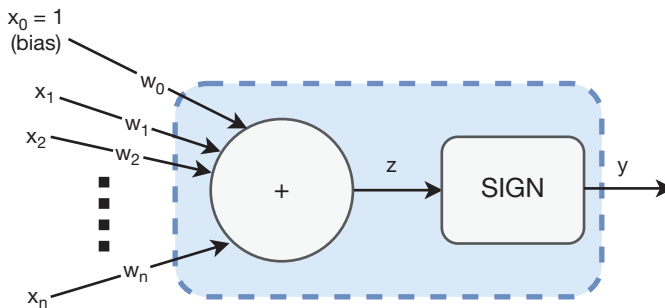


Figure 1-2 The perceptron

1. Hereafter, we do not discuss biological neurons, so any future reference to a neuron refers to an *artificial* neuron. Further, we often refer to a perceptron as a neuron because the perceptron is just a special type of neuron, and we often prefer the more generic name neuron except for when we detail properties that apply only to the perceptron.

2. The signum function should not be confused with the sigmoid function that is used for other neurons than perceptrons, as described in later chapters.

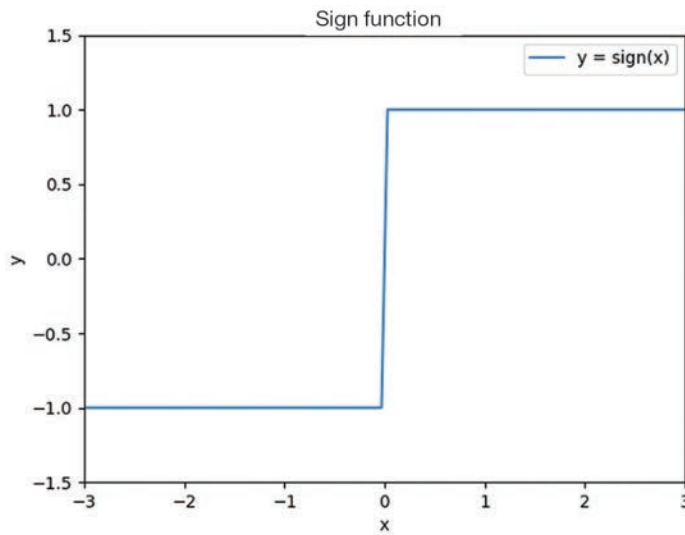


Figure 1-3 Sign (or signum) function. The figure uses variable names typically used for generic functions (y is a function of x). In our perceptron use case, the input to the signum function is not x but the weighted sum z .

A **perceptron** is a type of **artificial neuron**. It sums up the inputs to compute an intermediate value z , which is fed to an **activation function**. The perceptron uses the **sign function** as an activation function, but **other artificial neurons use other functions**.

To summarize, the perceptron will output -1 if the weighted sum is less than zero, and otherwise it will output 1 . Written as an equation, we have the following:

$$y = f(z), \text{ where}$$

$$z = \sum_{i=0}^n w_i x_i$$

$$f(z) = \begin{cases} -1, & z < 0 \\ 1, & z \geq 0 \end{cases}$$

$$x_0 = 1 \text{ (bias term)}$$

We note how the bias term x_0 is special in that it always is assigned the value 1. Its corresponding weight w_0 is treated just like any other weight. Code Snippet 1-1 implements this function programmatically in Python. The first element of x represents the bias term and thus must be set to 1 by the caller of the function.

Code Snippet 1-1 Python Implementation of Perceptron Function

```
# First element in vector x must be 1.
# Length of w and x must be n+1 for neuron with n inputs.
def compute_output(w, x):
    z = 0.0
    for i in range(len(w)):
        z += x[i] * w[i] # Compute sum of weighted inputs
    if z < 0: # Apply sign function
        return -1
    else:
        return 1
```

We said in the preface that you should learn Python, so if you have not yet, now is a good time for you to start going over that Python tutorial.

At this point, the special bias input might seem odd, but we show later in this chapter how varying the bias weight is equivalent to adjusting the threshold at which the perceptron changes its output value.

Example of a Two-Input Perceptron

A simple example provides an idea of how the perceptron works in practice. Let us study a perceptron with two inputs in addition to the bias input. Without any justification (at this point), we set the weights to $w_0 = 0.9$, $w_1 = -0.6$, and $w_2 = -0.5$. See Figure 1-4.

Now let us see how this perceptron behaves for all input combinations assuming that each of the two inputs can take on only the values -1.0 and 1.0 . If you want to get your hands dirty, you can paste Code Snippet 1-1 into a Python interpreter

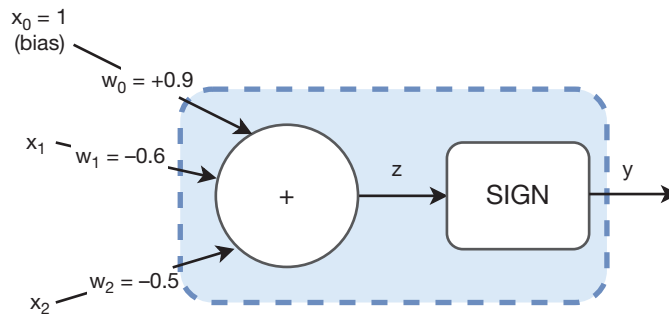


Figure 1-4 Perceptron with two inputs (in addition to the bias input) and defined weights

window (see Appendix I) and then call the function with the chosen weights and different x -input combinations. Remember that the first x -input should always be 1.0 because it represents the bias term. You should end up with the following if you call the function four times with all different combinations of x -inputs:

```
>>> compute_output([0.9, -0.6, -0.5], [1.0, -1.0, -1.0])
1
>>> compute_output([0.9, -0.6, -0.5], [1.0, -1.0, 1.0])
1
>>> compute_output([0.9, -0.6, -0.5], [1.0, 1.0, -1.0])
1
>>> compute_output([0.9, -0.6, -0.5], [1.0, 1.0, 1.0])
-1
```

To explore this behavior in more detail, we show the four different combinations in Table 1-1.

The table shows the inputs and the outputs, the intermediate values after applying the weights, as well as the sum before applying the activation function. Note what happens if we interpret the inputs and outputs as Boolean values, where -1 represents `False` and $+1$ represents `True`. The perceptron with these specific weights implements a NAND gate! Paraphrasing Nielsen, this is comforting because we know that by combining multiple NAND gates, we can build any logical function, but it is also kind of disappointing because we thought that neural networks were going to be something much more exciting than just Boolean logic (Nielsen, 2015).

Table 1-1 Behavior of a Perceptron with Two Inputs*

X_0	X_1	X_2	$W_0 * X_0$	$W_1 * X_1$	$W_2 * X_2$	Z	Y
1	-1 (False)	-1 (False)	0.9	0.6	0.5	2.0	1 (True)
1	1 (True)	-1 (False)	0.9	-0.6	0.5	0.8	1 (True)
1	-1 (False)	1 (True)	0.9	0.6	-0.5	1.0	1 (True)
1	1 (True)	1 (True)	0.9	-0.6	-0.5	-0.2	-1 (False)

*The values of the inputs and output can also be interpreted as Boolean values.

We said in the Preface that we would avoid starting with traditional machine learning techniques in favor of jumping straight to the more recent stuff, and now we start talking about basic binary logic gates instead. Not cool! Seems like an epic fail, but bear with us. At least we are already talking about neurons, and we will soon move on to more impressive ways of using them.

As we soon will see, it turns out that neural networks are different from Boolean logic. This difference becomes clear in later chapters, where we demonstrate the amazing things that neural networks can achieve. Meantime, we can list some specific differences:

- Perceptron inputs are not limited to Boolean values. In addition, although perceptrons are limited to outputting only one of two values, other neuron models can output a range of real numbers.
- In our simple example, the perceptron has only two inputs and implements a basic logical function. In the networks we study later in the book, each neuron has many more inputs, often more than what is typical for a logic gate. Each neuron can also implement more complex functions than AND and OR.
- We know of a learning algorithm that can be used to automatically design neural networks by learning from examples. Curiously, the resulting networks

tend to generalize and learn behavior that makes sense for not-yet-observed examples (this statement may seem fairly abstract, but the next section introduces the perceptron learning algorithm, and Chapter 4, “Fully Connected Networks Applied to Multiclass Classification,” discusses generalization).

The Perceptron Learning Algorithm

In the previous example, we somewhat arbitrarily picked the three weights, and we ended up with a perceptron that behaves like a NAND gate if we view the inputs as Boolean values. By inspecting Table 1-1, it should be fairly easy to convince yourself that the chosen weights are not the only ones that result in this outcome. For example, you can see that the z -value is far enough from zero in all cases, so you should be able to adjust one of the weights by 0.1 in either direction and still end up with the same behavior. This raises the questions of how we came up with these weights in the first place and whether there is a general approach for determining the weights. This is where the perceptron learning algorithm comes into play.

We first describe the algorithm itself and apply it to a couple of problems. These experiments provide some understanding of how the algorithm works but also reveal some of the limitations of the perceptron. We then show that it is possible to overcome these limitations and examine the perceptron from other angles. In Chapter 2, “Gradient-Based Learning,” we describe a somewhat more formal reasoning behind what the algorithm does.

The perceptron learning algorithm is what is called a *supervised learning algorithm*. The notion of supervision implies that the *model* that is being trained (in this case, the perceptron) is presented with both the input data and the desired output data (also known as *ground truth*). Think of it as a teacher presenting the question and answer to the model with the expectation that the model will learn that a certain input is associated with a corresponding output. The opposite of supervised learning is unsupervised learning in which the learning algorithm is responsible for finding patterns in the data by itself. An example of this concept is an algorithm that can find structure in natural language text. We study this concept in more detail in Chapter 11, “Text Autocompletion with LSTM and Beam Search,” where we train a model to do autocompletion of text.

The term **model** is often used as a synonym for a **network**. That is, when we talk about training a model, it is the same thing as coming up with weights for a network consisting of one or more neurons.

In our example, we have four sets of input/output data, each corresponding to one row in Table 1-1. The algorithm works as follows:

1. Randomly initialize the weights.
2. Select one input/output pair at random.
3. Present the values $x_1, \dots, x_n$ to the perceptron to compute the output y .
4. If the output y is different from the ground truth for this input/output pair, adjust the weights in the following way:
 - a. If $y < 0$, add ηx_i to each w_i .
 - b. If $y > 0$, subtract ηx_i from each w_i .
5. Repeat steps 2, 3, and 4 until the perceptron predicts all examples correctly.

The perceptron has certain limitations to what it can predict, so for some sets of input/output pairs, the algorithm will not converge. However, if it is possible to come up with a set of weights that enables the perceptron to represent the set of input/output pairs, then the algorithm is guaranteed to converge by finding these weights. The arbitrary constant η is known as the *learning rate*³ and can be set to 1.0, but setting it to a different value can lead to faster convergence of the algorithm. The learning rate is an example of a *hyperparameter*, which is not a parameter that is adjusted by the learning algorithm but can still be adjusted. For a perceptron, the weights can be initialized to 0, but for more complex neural networks, that is a bad idea. Therefore, we initialize them randomly to get into that habit. Finally, in step 4, it might seem like all the weights will be adjusted by the same amount, but remember that the input x_i is not limited to take on the two values -1 and 1 . It could well be 0.4 for one input and 0.9 for another, so the actual weight adjustment will vary.

We now walk through a Python implementation of this algorithm and apply it to our NAND example. Code Snippet 1-2 shows the initialization code where we first import a library for randomization and then initialize variables for the training examples and perceptron weights.

3. Some descriptions of the perceptron learning algorithm do not include the learning rate parameter, but since learning rate is an important parameter for the learning algorithm used for more complicated networks, we choose to introduce it here.

Code Snippet 1-2 Initialization Code for Our Perceptron Learning Example

```

import random

def show_learning(w):
    print('w0 =', '%5.2f' % w[0], ', w1 =', '%5.2f' % w[1],
          ', w2 =', '%5.2f' % w[2])

# Define variables needed to control training process.
random.seed(7) # To make repeatable
LEARNING_RATE = 0.1
index_list = [0, 1, 2, 3] # Used to randomize order

# Define training examples.
x_train = [(1.0, -1.0, -1.0), (1.0, -1.0, 1.0),
            (1.0, 1.0, -1.0), (1.0, 1.0, 1.0)] # Inputs
y_train = [1.0, 1.0, 1.0, -1.0] # Output (ground truth)

# Define perceptron weights.
w = [0.2, -0.6, 0.25] # Initialize to some "random" numbers

# Print initial weights.
show_learning(w)

```

Note how each input example consists of three values, but the first value is always 1.0 because it is the bias term. Code Snippet 1-3 restates the perceptron output computation that was shown in Code Snippet 1-1.

Code Snippet 1-3 Perceptron Function as Shown in Code Snippet 1-1

```

# First element in vector x must be 1.
# Length of w and x must be n+1 for neuron with n inputs.
def compute_output(w, x):
    z = 0.0
    for i in range(len(w)):
        z += x[i] * w[i] # Compute sum of weighted inputs
    if z < 0: # Apply sign function
        return -1
    else:
        return 1

```

Code Snippet 1-4 contains the perceptron training loop. It is a nested loop in which the inner loop runs through all four training examples in random order. For each example, it computes the output and adjusts and prints the weights if the output is wrong. The weight adjustment line contains a subtle detail that makes it look slightly different than how we described the algorithm. Instead of using an `if` statement to determine whether to use addition or subtraction to adjust the weights, the adjustment value is multiplied by `y`. The value of `y` will either be `-1` or `+1`, and consequently results in selecting between addition and subtraction for the update. The outer loop tests whether the perceptron provided correct output for all four examples and, if so, terminates the program.

Code Snippet 1-4 Perceptron Training Loop

```
# Perceptron training loop.
all_correct = False
while not all_correct:
    all_correct = True
    random.shuffle(index_list) # Randomize order
    for i in index_list:
        x = x_train[i]
        y = y_train[i]
        p_out = compute_output(w, x) # Perceptron function

        if y != p_out: # Update weights when wrong
            for j in range(0, len(w)):
                w[j] += (y * LEARNING_RATE * x[j])
            all_correct = False
            show_learning(w) # Show updated weights
```

If we paste the three snippets together into a single file and then run it in a Python interpreter, the output will look something like the following:

```
w0 = 0.20 , w1 = -0.60 , w2 = 0.25
w0 = 0.30 , w1 = -0.50 , w2 = 0.15
w0 = 0.40 , w1 = -0.40 , w2 = 0.05
w0 = 0.30 , w1 = -0.50 , w2 = -0.05
w0 = 0.40 , w1 = -0.40 , w2 = -0.15
```

Note how the weights are gradually adjusted from the initial values to arrive at weights that produce the correct output. Most code examples in this book make use of random values, so your results might not exactly match our results.

In addition to the described Python implementation, we also provide a spreadsheet that performs the same calculations. We find that directly modifying weights and input values in a spreadsheet is often a good way to build intuition. The location from which to download the spreadsheet can be found in the programming examples section in Appendix I.

Now that we have seen that this algorithm can learn the NAND function, we will explore a little bit more in depth what it learned. Up until now, we have restricted ourselves to making each input take on just one of two values (either -1 or 1). However, there is nothing that prevents us from presenting any real number on the two inputs. That is, we can present any combination of two real numbers to the perceptron, and it will produce either -1 or 1 on its output. One way to illustrate this is to make a chart of a 2D coordinate system where one axis represents the first input (x_1), and the other axis represents the second input (x_2). For each point in this coordinate system, we can write a “+” or a “-” depending on what value the perceptron outputs. Such a chart is plotted in Figure 1-5.

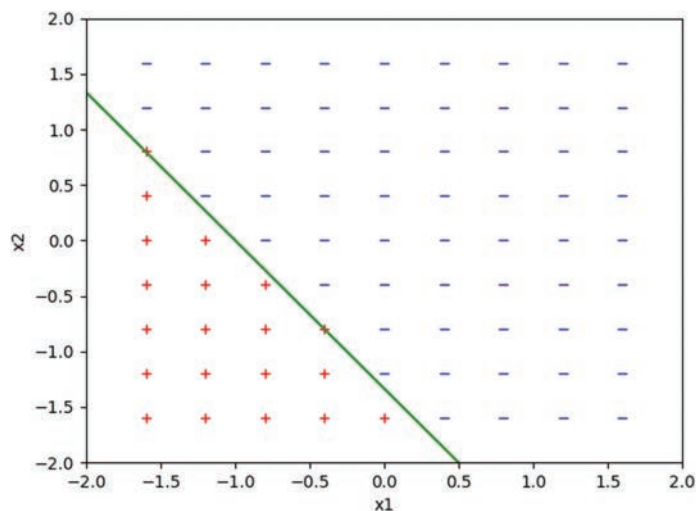


Figure 1-5 Output of a perceptron as a function of two inputs x_1 and x_2

This chart is different than plotting the function $y = f(x_1, x_2)$ in a traditional manner. A traditional plot of a function that takes two values as inputs and produces one value as an output would produce a 3D chart where some kind of surface is plotted as a function of the two inputs (this is shown later under “Geometrical Interpretation of the Perceptron”). Figure 1-5 is different in that instead of plotting the output value on its own axis (the z-axis in a 3D chart), we simply show the numeric values as symbols (+ and -) on the chart, which is simple to do because there are only two possible values (-1 and 1), and they also happen to fall in a nice simple pattern.

As you can see from the figure, the perceptron divides the 2D space into two regions, separated by a straight line, where all input values on one side of the line produce the output -1, and all input values on the other side of the line produce the output +1. A natural question is how we came up with the chart in the first place. One brute-force way of doing this is to test all combinations of (x_1, x_2) pairs and record the output from the perceptron. For the purpose of this discussion, this would be a fine way to do things, but if you are interested, it is simple to derive the equation for the line that separates the two regions. We know that the line represents the boundary between negative and positive output values of the perceptron. This boundary is exactly where the weighted sum of the inputs is zero, because the sign function will change its value when its input is zero. That is, we have

$$w_0x_0 + w_1x_1 + w_2x_2 = 0$$

We want to rewrite this equation so that x_2 is a function of x_1 , because x_2 is plotted on the y-axis, and normally when plotting a straight line, we do $y = f(x)$. We insert 1 for x_0 , solve the equation for x_2 , and arrive at

$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{w_0}{w_2}$$

In other words, it is a straight line with slope $-w_1/w_2$ and a y-intercept of $-w_0/w_2$.

Now that we are familiar with this type of chart, we can look at the learning process in more detail. We replace the initialization code in our program by the extended version shown in Code Snippet 1-5. In this code snippet, we have extended the `show_learning()` function to produce a plot like the one described previously. In addition to the changes in Code Snippet 1-5, we need to add the following code line at the end of the program:

```
plt.show()
```

We do not describe details of the plot routine because it is uninteresting and is not built upon later in the book.

Code Snippet 1-5 Extended Version of Initialization Code with Function to Plot the Output

```
import matplotlib.pyplot as plt
import random

# Define variables needed for plotting.
color_list = ['r-', 'm-', 'y-', 'c-', 'b-', 'g-']
color_index = 0

def show_learning(w):
    global color_index
    print('w0 =', '%5.2f' % w[0], ', w1 =', '%5.2f' % w[1],
          ', w2 =', '%5.2f' % w[2])
    if color_index == 0:
        plt.plot([1.0], [1.0], 'b-', markersize=12)
        plt.plot([-1.0, 1.0, -1.0], [1.0, -1.0, -1.0],
                  'r+', markersize=12)
        plt.axis([-2, 2, -2, 2])
        plt.xlabel('x1')
        plt.ylabel('x2')
    x = [-2.0, 2.0]
    if abs(w[2]) < 1e-5:
        y = [-w[1]/(1e-5)*(-2.0)+(-w[0]/(1e-5)),
              -w[1]/(1e-5)*(2.0)+(-w[0]/(1e-5))]
    else:
        y = [-w[1]/w[2]*(-2.0)+(-w[0]/w[2]),
              -w[1]/w[2]*(2.0)+(-w[0]/w[2])]
    plt.plot(x, y, color_list[color_index])
    if color_index < (len(color_list) - 1):
        color_index += 1

# Define variables needed to control training process.
random.seed(7) # To make repeatable
LEARNING_RATE = 0.1
index_list = [0, 1, 2, 3] # Used to randomize order
```

```

# Define training examples.
x_train = [(1.0, -1.0, -1.0), (1.0, -1.0, 1.0),
           (1.0, 1.0, -1.0), (1.0, 1.0, 1.0)] # Inputs
y_train = [1.0, 1.0, 1.0, -1.0] # Output (ground truth)

# Define perceptron weights.
w = [0.2, -0.6, 0.25] # Initialize to some "random" numbers

# Print initial weights.
show_learning(w)

```

The resulting plot is shown in Figure 1-6, where the four input points are shown as three plus signs and one minus sign. The red line corresponds to the initial set of weights that do not correctly divide the chart between the plus and minus signs. For each weight update, we plot another line in the following color order: magenta, yellow, cyan, and blue. The blue line correctly divides the chart with all plus signs on one side and the minus sign on the other side, so the learning algorithm terminates.

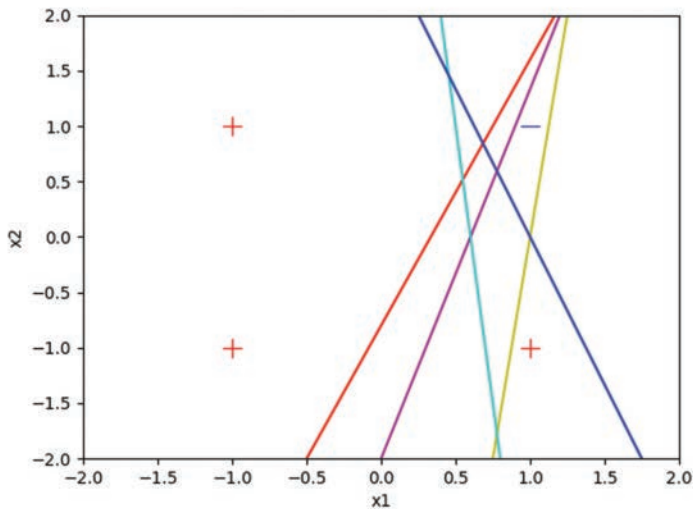


Figure 1-6 Learning process progressing in the following order: red, magenta, yellow, cyan, blue

We have now shown that the perceptron can learn to do a simple classification task, namely, to determine if a two-value input pair belongs to one class or another. It is not as advanced as distinguishing between a dog and a cat, but we need to learn to walk before we can run.

Limitations of the Perceptron

In Chapter 2, we look at the learning algorithm in more detail to justify why it works. However, you might have noticed⁴ that we just ran into a big limitation of the perceptron. Let us take a moment to understand this limitation and its implications.

We saw that the two-input perceptron learns how to draw a straight line between two groups of data points. That is exciting, but what happens if a straight line cannot separate the data points? We explore this scenario using a different Boolean function, namely, the exclusive OR, also known as XOR. Its truth table is shown in Table 1-2.

Figure 1-7 shows these four data points on the same type of chart that we studied before, illustrating how the algorithm tries to learn how to draw a line between the plus and minus signs. The top chart shows what it looks like after 6 weight updates and the bottom chart, after 30 weight updates—where we have also run out of colors and the algorithm never converges.

It is trivial to solve the problem with a curved line but is not possible with a straight line. This is one of the key limitations of the perceptron. It can solve

Table 1-2 Truth Table for a Two-Input XOR Gate

X_0	X_1	Y
False	False	False
True	False	True
False	True	True
True	True	False

4. It is perfectly fine if you did not notice this. It is far from obvious when you see it the first time.

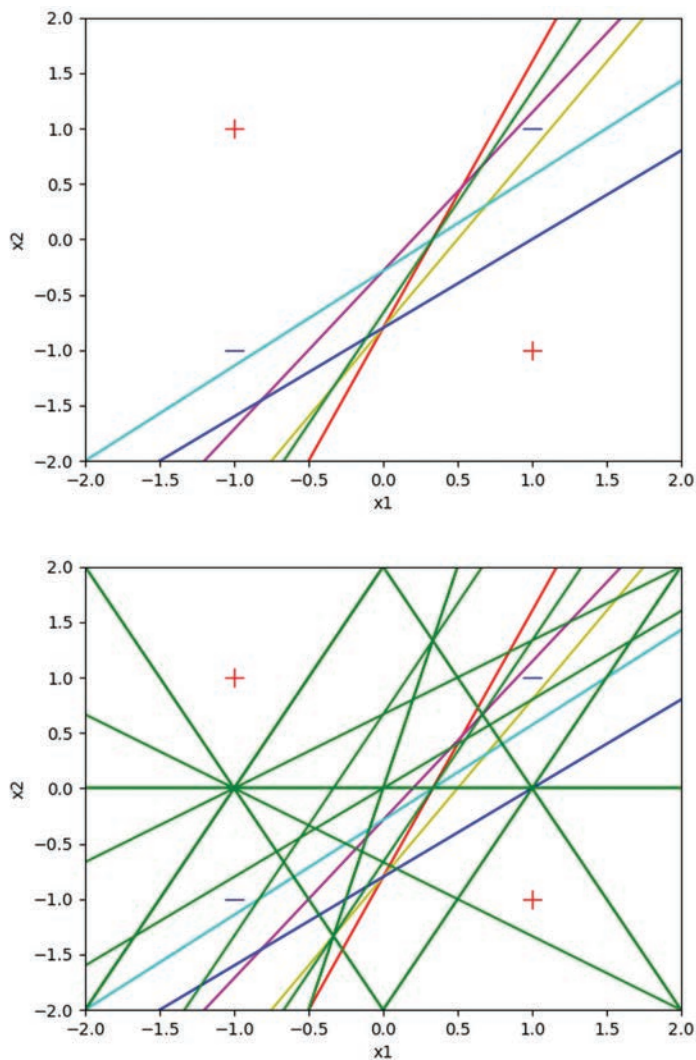


Figure 1-7 Perceptron attempting to learn XOR. Top: After 6 weight adjustments. Bottom: After 30 weight adjustments.

classification problems only where the classes are linearly separable, which in two dimensions (two inputs) means that the data points can be separated by a straight line. Thus, it seems we need to either come up with a different model of a neuron or combine multiple of them to solve the problem. In the next section, we explore the latter solution.

Combining Multiple Perceptrons

As shown previously, a single perceptron can separate the chart into two regions, illustrated by drawing a straight line on the chart. That means that if we add another perceptron, we can draw another straight line. Figure 1-8 shows one such attempt: One line separates one of the minuses from all other data points. Similarly, the other line separates the other minus also from all other data points. If we somehow can output 1 only for the data points between the two lines, then we have solved the problem.

Another way to look at it is that each of the two perceptrons will fire correctly for three out of four data points; that is, both of them almost do the right thing. They both incorrectly categorize one data point, but not the same one. If we could combine the output of the two, and output 1 only when both of them compute the output as a 1, then we would get the right result. So, we want to do an AND of their outputs, and we know how to do that. We just add yet another perceptron that uses the outputs of the two previous perceptrons as its inputs. The architecture of this two-level neural network and the weights are shown in Figure 1-9.

Table 1-3 shows the output of each of the three neurons. Looking at x_1 , x_2 , and y_2 , it is clear that the neural network implements the XOR function.

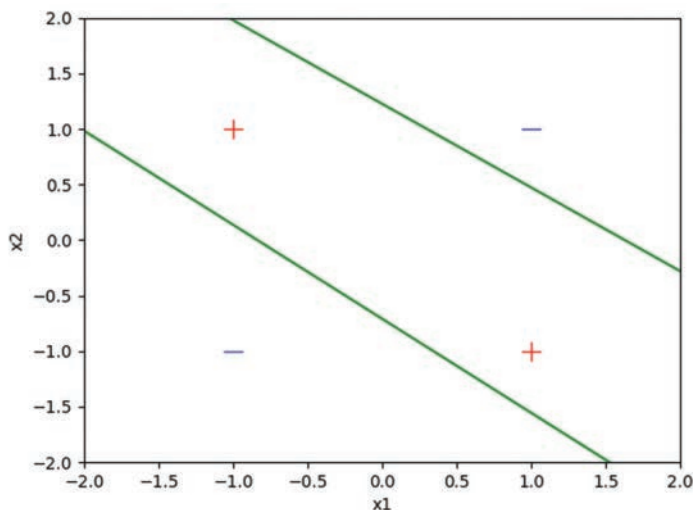


Figure 1-8 XOR output values isolated by two lines

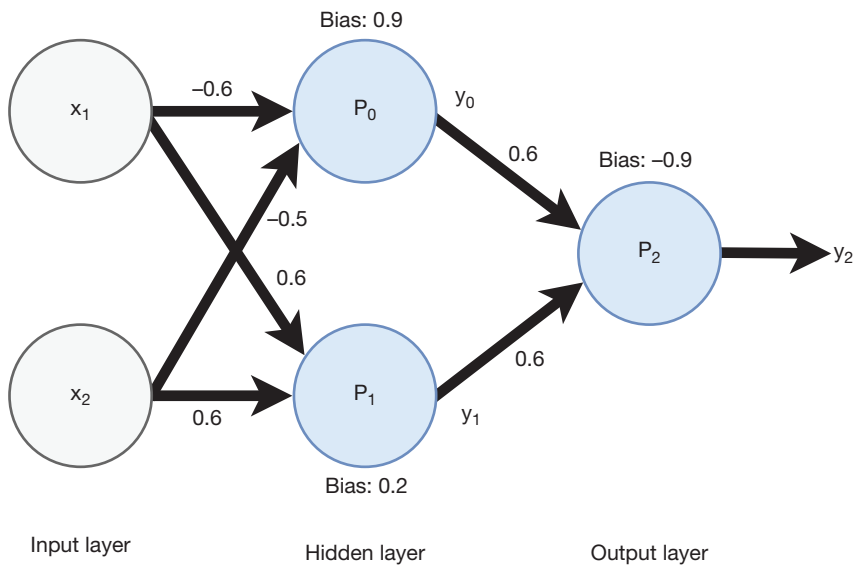


Figure 1-9 Two-level feedforward network implementing XOR function

Table 1-3 Input and Output Values Showing That the Network Implements the XOR Function

x_0	x_1	x_2	y_0	y_1	y_2
1	-1 (False)	-1 (False)	1.0	-1.0	-1.0 (False)
1	1 (True)	-1 (False)	1.0	1.0	1.0 (True)
1	-1 (False)	1 (True)	1.0	1.0	1.0 (True)
1	1 (True)	1 (True)	-1.0	1.0	-1.0 (False)

This neural network is one of the simplest examples of a fully connected feedforward network. *Fully connected* means that the output of each neuron in one layer is connected to all neurons in the next layer. *Feedforward* means that there

are no backward connections, or, using graph terminology, it is a directed acyclic graph (DAG). The figure also highlights the concept of layers. A multilevel neural network has an input layer, one or more hidden layers, and an output layer. The input layer does not contain neurons but contains only the inputs themselves; that is, there are no weights associated with the input layer. Note that the single perceptron we looked at also had an input layer, but we did not explicitly draw it. In Figure 1-9, the output layer has only one neuron, but in general, the output layer can consist of more than one neuron. Similarly, the network in the figure has only a single hidden layer (with two neurons), but a deep neural network (DNN) has more than one hidden layer and typically many more neurons in each layer. The weights (including bias) in the figure are spelled out, but in most cases, it is just assumed that they are there, and they are not shown. A feedforward network is also known as a *multilevel perceptron* even when it is built from neuron models that are not perceptrons, which can be somewhat confusing.

In a **fully connected network**, a neuron in one layer receives inputs from all other neurons in the immediately preceding layer. A **feedforward network** or **multilevel perceptron** has no cycles. The **input layer** has no neurons. The outputs of neurons in a **hidden layer** are not visible (they are hidden) outside of the network. DNNs have **multiple hidden layers**. The **output layer** can have multiple neurons.

This XOR example is starting to get close to our definition of deep learning (DL): *DL is a class of machine learning algorithms that use multiple layers of computational units where each layer learns its own representation of the input data. These representations are combined by later layers in a hierarchical fashion.* In the preceding example, we did have multiple (two) layers. The neurons in the first layer had their own representation (the output from the hidden layer) of the input data, and these representations were combined hierarchically by the output neuron. A missing piece is that *each layer learns its own representation*. In our example, the network did not learn the weights, but we came up with them. A valid question is how we came up with all these weights. The answer is that we picked them carefully. The weights for the first perceptron already implemented a NAND function. We then picked weights for the second perceptron in the first layer to implement an OR function, and finally we picked the weights for the perceptron in the second layer to implement an AND function. By doing this, we arrived at the Boolean function for XOR:

$$\overline{(A \cdot B)} \cdot (A + B)$$

Although we are just assuming -1 and $+1$ as input values in this discussion, the neural network that we have created can take any real numbers as inputs, and it will output 1 for all the points between the two lines in Figure 1-8. We chose the weights carefully to make the neural network behave the way we wanted, which was possible for this specific example but is nontrivial for the general case. Is there such an algorithm for a multilevel neural network? As described in the Preface, Minsky and Papert (1969) did not think so. However, it turns out that history proved the doubters wrong. The back-propagation algorithm was applied to various problem types from at least 1970 (Linnainmaa, 1970) and was popularized for neural networks in 1986 (Rumelhart, Hinton, and Williams, 1986). We cover this algorithm in detail in Chapter 3, “Sigmoid Neurons and Back-Propagation,” but first we explore the perceptron a little bit more.

Implementing Perceptrons with Linear Algebra

Knowledge of linear algebra is handy when working with neural networks. We now introduce some basic concepts and describe how they relate to the perceptron and why this knowledge is useful. In particular, we show how we can describe the input examples and perceptron weights as vectors and matrices and how parts of the perceptron calculations are equivalent to dot products, matrix-vector multiplications, and matrix multiplications. Computing dot products, matrix-vector multiplications, and matrix multiplications efficiently is important in many scientific fields, so much effort has been spent on creating efficient implementations of these operations. For example, if you program in Python, there is a package known as NumPy, which is used for scientific computations. It has specific functions for the above-mentioned operations. Under the hood, NumPy makes use of the Basic Linear Algebra Subprograms (BLAS), which is heavily optimized to run as fast as possible on the platform where it is run. Further, if you have a graphics processing unit (GPU) capable of running CUDA, there is the CUDA BLAS (cuBLAS) library that enables NumPy to perform these operations efficiently on the GPU,⁵ which can give you orders of magnitude speedup compared to running on a CPU.

5. Modern DL frameworks make use of a library known as cuDNN, which is specifically developed to accelerate neural network computations by offloading them to the GPU.

Specifying mathematical problems in **vector** or **matrix form** enables you to take advantage of efficient mathematics library implementations, and in particular to **offload computations to the GPU**.

If you already know linear algebra, you will find that much of this section describes basic mathematical concepts. If this is the case, you should focus on how the perceptron weights and inputs map to these basic concepts. If you have some linear algebra knowledge but have not used these skills for a while, we think this section will serve as a good refresher. If you have never seen linear algebra before, then we recommend that you start with reading this section. This is likely sufficient for many readers, whereas others might want to pursue more in-depth descriptions of the topic.

VECTOR NOTATION

In the previous sections, we saw examples of a large number of variables, such as multiple inputs (x) and corresponding weights (w) for each neuron, intermediate representations (z), and outputs (y). Common for all these variables is that they consist of a single value, which also is known as a *scalar variable*. It is often the case that we have a number of variables that kind of belong together, such as all the input variables $x_0, x_1, \dots, x_n$ for a single neuron. One way to look at things is that each of these individual variables is only one component of the overall input. A more compact notation to use in this case is to arrange these scalar values into a vector variable:

$$x = \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{pmatrix}$$

We start variable subscripts at 0 because that is how it is done in most programming languages. The typical convention in linear algebra is to start indexing at 1, but we will continue using 0 to stay consistent and avoid any confusion when translating formulas into code.

Similarly, we can arrange the weight variables into a single weight vector variable:

$$\mathbf{w} = \begin{pmatrix} w_0 \\ w_1 \\ \vdots \\ w_n \end{pmatrix}$$

We use lowercase ***bold italic*** letters to denote vectors in this book.

If you are a programmer, then a vector should be a familiar concept, although it is typically known as an array. The vectors shown so far are also known as *column vectors*, because they are arranged vertically as a column. We can also arrange the elements horizontally into a *row vector*. We use the *transpose* operation to convert a column vector into a row vector. The vector $\mathbf{x}$ and its transpose are shown here:

$$\mathbf{x} = \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{pmatrix}, \quad \mathbf{x}^T = \begin{pmatrix} x_0 & x_1 & \dots & x_n \end{pmatrix}$$

In the field of linear algebra, mathematical operations have been defined for vectors and other related structures. One example is vector addition, which can be used to add one vector to another vector with the same number of elements. Vector addition is an element-wise operation. Element 0 of both vectors are added together to form element 0 of the output vector, element 1 of both vectors are added together to form element 1 of the output vector, and so on:

$$\mathbf{a} = \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} \quad \mathbf{b} = \begin{pmatrix} b_0 \\ b_1 \\ \vdots \\ b_n \end{pmatrix} \quad \mathbf{a} + \mathbf{b} = \begin{pmatrix} a_0 + b_0 \\ a_1 + b_1 \\ \vdots \\ a_n + b_n \end{pmatrix}$$

All of this enables a compact way of describing operations on the elements in the vectors. This is just an abstraction. We hide the individual elements when writing down the equations, but when we need to perform the computations, we are still working on each individual value (although, as previously mentioned, some hardware implementations can do the operations efficiently in parallel).

DOT PRODUCT

Another important operation on two vectors is the *dot product*. The dot product is defined only if the two vectors are of equal length, just like vector addition. It is computed by multiplying element 0 of the two vectors, then multiplying element 1 of the two vectors and so on, and finally adding all of these products:

$$\mathbf{w} \cdot \mathbf{x} = w_0x_0 + w_1x_1 + \dots + w_nx_n = \sum_{i=0}^n w_ix_i$$

These computations might seem familiar. It is exactly how we compute the weighted sum z in our perceptron. That is, assuming that we have arranged the inputs into a vector $\mathbf{x}$ (where the first element is 1) and the weights into a vector $\mathbf{w}$ (where the first element is the bias weight), then we can write the perceptron equation as

$$y = \text{sign}(\mathbf{w} \cdot \mathbf{x})$$

It is almost as if the dot product was invented to implement perceptrons. Just as with the vector addition, the dot product did not eliminate any of the computations but only simplified the notation. However, in practice, it also enabled us to call an efficient library implementation of the computation instead of a loop-based implementation of the weighted sum. Code Snippet 1-6 shows how we can write our perceptron function using the NumPy dot-product functionality. We also changed the code to use the NumPy sign function instead of implementing it with an `if` statement.

Code Snippet 1-6 Our Perceptron Function Rewritten Using Vector Notation

```
import numpy as np
def compute_output_vector(w, x):
    z = np.dot(w, x)
    return np.sign(z)
```

EXTENDING THE VECTOR TO A 2D MATRIX

The vector concept is a special case of the more general concept of a multidimensional structure where the dimension of a vector is 1. A multidimensional structure in two dimensions is known as a *matrix* and is described next. An example of a matrix A with $m+1$ rows and $n+1$ columns is shown here:⁶

$$A = \begin{pmatrix} a_{00} & a_{01} & \dots & a_{0n} \\ a_{10} & a_{11} & \dots & a_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m0} & a_{m1} & \dots & a_{mn} \end{pmatrix}$$

We use italic uppercase letters to denote matrices in this book.

The numbering of elements in a matrix is somewhat different than if you refer to coordinates in a 2D coordinate system (an xy chart). In particular, the elements in the vertical direction are numbered in increasing order going downward in a matrix, whereas an increasing y -value in an xy chart is increasing going upward. In addition, in an xy chart, we state the horizontal coordinate (x) first and the vertical coordinate (y) second, whereas for a matrix, we state the row first and the column second. As an example, element a_{01} is the top element second from the left in a matrix, and the coordinate ($x=0, y=1$) is the leftmost element and second from the bottom in an xy chart.

Why do we want to use this 2D structure when working with neurons? It is seldom the case that we work with a single neuron or a single input example. We just saw how the weights ($\mathbf{w}$) for a single neuron can be represented by a vector. This means that we can represent the weights for n neurons with n vectors by arranging them in a matrix. Similarly, we seldom work with a single input example ($\mathbf{x}$), but we have a whole set of input examples. Just as for multiple neurons, we can represent a set of input examples as a set of vectors that can be arranged in a matrix structure.

Just as we can transpose a vector, we can also transpose a matrix. We do this by flipping the matrix along its diagonal; that is, element ij , where i is the

6. Just as we do for vectors, in this book we start with a subscript of 0 for matrices to keep it consistent with programming in Python. The convention in mathematical texts on the topic is to start with a subscript of 1.

column and j is the row, becomes element ji . An example for a 2×2 matrix is shown here:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad A^T = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$$

Now that we know the basics of the matrix, we are ready to move on to some important matrix operations.

MATRIX-VECTOR MULTIPLICATION

Using the preceding concepts, we are now ready to define *matrix-vector multiplication*:

$$\mathbf{y} = \mathbf{Ax} = \begin{pmatrix} a_{00} & a_{01} & \dots & a_{0n} \\ a_{10} & a_{11} & \dots & a_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m0} & a_{m1} & \dots & a_{mn} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} a_{00}x_0 + a_{01}x_1 + \dots + a_{0n}x_n \\ a_{10}x_0 + a_{11}x_1 + \dots + a_{1n}x_n \\ \vdots \\ a_{m0}x_0 + a_{m1}x_1 + \dots + a_{mn}x_n \end{pmatrix}$$

It is defined only for cases where the number of columns in the matrix matches the number of elements in the vector. It results in a vector with the same number of elements as there are rows in the matrix. The elements in this resulting vector are defined as

$$y_i = \sum_{j=0}^n a_{ij}x_j$$

We recognize the sum as the dot product between two vectors, as previously described; that is, a slightly different view of the matrix is to consider each of the $m+1$ rows of the matrix as row vectors (a transposed vector). In that case, the matrix-vector multiplication can be viewed as doing $m+1$ dot products of the matrix rows and the x -vector, as shown here:

$$\mathbf{y} = \mathbf{Ax} = \begin{pmatrix} \mathbf{a}_0^T \\ \mathbf{a}_1^T \\ \vdots \\ \mathbf{a}_m^T \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} \mathbf{a}_0^T \cdot \mathbf{x} \\ \mathbf{a}_1^T \cdot \mathbf{x} \\ \vdots \\ \mathbf{a}_m^T \cdot \mathbf{x} \end{pmatrix}$$

Now let us look at how to use matrix-vector multiplication in the context of perceptrons. Assume that we have $m+1$ perceptrons, each having n inputs plus the bias input. Further, we have a single input example consisting of $n+1$ values, where the first element in the input vector is 1 to represent the bias input value. Now assume that we arrange the vectors for the perceptrons' weights into a matrix W , so we have

$$W = \begin{pmatrix} \mathbf{w}_0^T \\ \mathbf{w}_1^T \\ \vdots \\ \mathbf{w}_m^T \end{pmatrix}$$

where each $\mathbf{w}_i$ is a multielement vector corresponding to a single neuron. We can now compute the weighted sums for all $m+1$ perceptrons for the input example $\mathbf{x}$ by multiplying the matrix by the vector:

$$\mathbf{z} = W\mathbf{x}$$

The vector $\mathbf{z}$ will now contain $m+1$ elements, where each element represents the weighted sum for a single neuron presented with the input example.

MATRIX-MATRIX MULTIPLICATION

Let us now introduce *matrix-matrix multiplication* of two matrices, A and B :

$$C = AB =$$

$$\begin{pmatrix} a_{00} & a_{01} & \dots & a_{0n} \\ a_{10} & a_{11} & \dots & a_{1n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m0} & a_{m1} & \dots & a_{mn} \end{pmatrix} \begin{pmatrix} b_{00} & b_{01} & \dots & b_{0p} \\ b_{10} & b_{11} & \dots & b_{1p} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n0} & b_{n1} & \dots & b_{np} \end{pmatrix} = \begin{pmatrix} c_{00} & c_{01} & \dots & c_{0p} \\ c_{10} & c_{11} & \dots & c_{1p} \\ \vdots & \vdots & \ddots & \vdots \\ c_{m0} & c_{m1} & \dots & c_{mp} \end{pmatrix}$$

The number of columns in the first matrix A must match the number of rows in the second matrix B . The elements of the resulting matrix C are defined as

$$c_{ij} = a_{i0}b_{0j} + a_{i1}b_{1j} + \dots + a_{in}b_{nj}$$

or, alternatively,

$$c_{ij} = \sum_{k=0}^n a_{ik} b_{kj}$$

Again, we recognize this sum as a dot product. That is, if we view each of the $m+1$ rows of matrix A as a row vector and each of the $p+1$ columns of matrix B as a column vector, then the matrix multiplication results in $(m+1) \times (p+1)$ dot products. In other words, we compute all the dot products between all row-vectors in matrix A and all column-vectors in matrix B . To make this abundantly clear, we can write the definition in a slightly different form. We state the two matrices A and B as being collections of vectors, and the elements of the resulting matrix are computed as dot products between these vectors:

$$C = AB = \begin{pmatrix} \mathbf{a}_0^T \\ \mathbf{a}_1^T \\ \vdots \\ \mathbf{a}_m^T \end{pmatrix} \begin{pmatrix} \mathbf{b}_0 & \mathbf{b}_1 & \dots & \mathbf{b}_p \end{pmatrix} = \begin{pmatrix} \mathbf{a}_0 \cdot \mathbf{b}_0 & \mathbf{a}_0 \cdot \mathbf{b}_1 & \dots & \mathbf{a}_0 \cdot \mathbf{b}_p \\ \mathbf{a}_1 \cdot \mathbf{b}_0 & \mathbf{a}_1 \cdot \mathbf{b}_1 & \dots & \mathbf{a}_1 \cdot \mathbf{b}_p \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{a}_m \cdot \mathbf{b}_0 & \mathbf{a}_m \cdot \mathbf{b}_1 & \dots & \mathbf{a}_m \cdot \mathbf{b}_p \end{pmatrix}$$

All of this can be somewhat heavy to get through if you are not well versed in linear algebra, but familiarity with these notations will be very helpful for your future work with DL.

Similarly, to what we did with matrix-vector multiplication, we can use matrix-matrix multiplication in the context of perceptrons. Assume that we have $m+1$ perceptrons, each having n inputs (+ bias input) just as in the previous example. Further, we have $p+1$ input examples, each consisting of $n+1$ values. As always, we assume that the first element in each input vector is 1 to represent the bias input value. Now assume that in addition to the matrix W , which represents the perceptrons' weights, we arrange the vectors for the input examples into matrix X :

$$W = \begin{pmatrix} \mathbf{w}_0^T \\ \mathbf{w}_1^T \\ \vdots \\ \mathbf{w}_m^T \end{pmatrix}, \quad X = \begin{pmatrix} \mathbf{x}_0 & \mathbf{x}_1 & \dots & \mathbf{x}_p \end{pmatrix}$$

In this example, both w_i and x_i refer to multielement vectors. We can now compute the weighted sums for all $m+1$ perceptrons for all $p+1$ input examples by multiplying the two matrices:

$$Z = WX$$

The matrix Z will now contain $(m+1) \times (p+1)$ elements, where each element represents the weighted sum for a single neuron presented with a single input example. To make this more concrete, matrix W corresponds to the two neurons in the first layer of our XOR network previously shown in Figure 1-9. Matrix X contains all four input examples. The resulting matrix WX contains the weighted sum for the two neurons for all four input examples:

$$W = \begin{pmatrix} w_{00} & w_{01} & w_{02} \\ w_{10} & w_{11} & w_{12} \end{pmatrix} = \begin{pmatrix} 0.9 & -0.6 & -0.5 \\ 0.2 & 0.6 & 0.6 \end{pmatrix}$$

$$X = \begin{pmatrix} x_{00} & x_{01} & x_{02} & x_{03} \\ x_{10} & x_{11} & x_{12} & x_{13} \\ x_{20} & x_{21} & x_{22} & x_{23} \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ -1 & -1 & 1 & 1 \\ -1 & 1 & -1 & 1 \end{pmatrix}$$

$$WX = \begin{pmatrix} 2 & 1 & 0.8 & -0.2 \\ -1 & 0.2 & 0.2 & 1.4 \end{pmatrix}$$

The computation resulting in the value 2 for the upper left element in the resulting matrix is

$$w_{00}x_{00} + w_{01}x_{01} + w_{02}x_{02} = (0.9)(1) + (-0.6)(-1) + (-0.5)(-1) = 2$$

The other values can be computed following the same pattern. We simply compute the dot products between row vectors in W and column vectors in X .

SUMMARY OF VECTOR AND MATRIX OPERATIONS USED FOR PERCEPTRONS

The preceding discussion described how linear algebra operations map to various combinations of input examples and number of perceptrons. Table 1-4 summarizes the mappings.

None of the preceding exercises have simplified or eliminated any computations, but the notation enables computations to be computed efficiently in parallel on a GPU.

Table 1-4 Combinations of Perceptrons and Input Examples and the Corresponding Linear Algebra Operation

NUMBER OF PERCEPTRONS	NUMBER OF INPUT EXAMPLES	LINEAR ALGEBRA OPERATION
One	One	Dot product
Multiple	One	Matrix-vector multiplication
Multiple	Multiple	Matrix-matrix multiplication

DOT PRODUCT AS A MATRIX MULTIPLICATION

Before moving on from matrices and vectors, we want to introduce one more common notation. We note that a vector can be viewed as the special case of a matrix with a single column. This implies that we can formulate our dot product of two vectors as a matrix multiplication. Assume that we have vectors **a** and **b** as follows:

$$\mathbf{a} = \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} b_0 \\ b_1 \\ \vdots \\ b_n \end{pmatrix}$$

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=0}^n a_i b_i = \begin{pmatrix} a_0 & a_1 & \dots & a_n \end{pmatrix} \begin{pmatrix} b_0 \\ b_1 \\ \vdots \\ b_n \end{pmatrix} = \mathbf{a}^T \mathbf{b}$$

That is, if we transpose vector **a** so it becomes a matrix with a single row, we can now do matrix multiplication between **a^T** and **b** and thereby omit the dot product operator. This is a common notation with which it is good to be familiar.

EXTENDING TO MULTIDIMENSIONAL TENSORS

Vectors and matrices are special cases of the more generalized concept of a tensor, which is equivalent to the programming concept of a multidimensional

array. In other words, if we extended a matrix to another dimension, we would call the resulting entity a 3D tensor. Tensors can show up in cases where the input data itself is multidimensional, such as in a color image. It consists of a 2D array of pixel values, and each pixel consists of three components (red, green, and blue, or RGB for short). That is, the input data itself is 3D, and if we have a collection of images as input values, then we can organize all of these as a 4D tensor. This can be tricky at first and often takes some time to get used to. The greatest challenge is to keep track of all the indices correctly. In the end, all the computations are typically reduced to a large number of dot products.

Geometric Interpretation of the Perceptron

Previously in this chapter, we visualized the decision boundary created by a two-input perceptron. In that type of chart, we identified all the coordinates where the perceptron would output -1 and all the coordinates where the perceptron would output $+1$. Another way of visualizing what the perceptron does is to plot z as a function of x_1 and x_2 . This takes the form of a 3D chart, as shown in Figure 1-10.

We can see that the z -value in the perceptron forms a plane. The actual output (y) of the perceptron will take on the value of -1 for any point on the plane that is less than 0, whereas it will be $+1$ for any point that is greater than or equal to 0. If you look at this chart from above and draw a line where the z -value of the plane

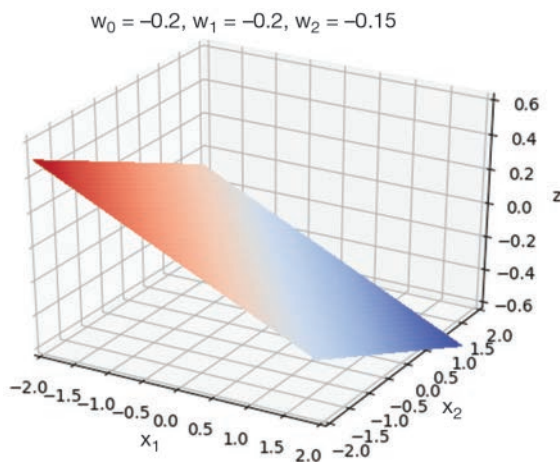


Figure 1-10 3D plot of two-input perceptron decision surface

is 0, you will end up with the same type of chart that we previously looked at in Figure 1-5.

The positioning and the orientation of the plane are determined by the three weights. The bias weight (w_0) determines where the plane will cross the z -axis at the point where both x_1 and x_2 equal 0; that is, changing w_0 causes the plane to move up or down on the z -axis. This is shown in the two upper charts in Figure 1-11, where the left chart is the same as in Figure 1-10, but in the right chart, we changed w_0 from -0.2 to 0.0 . The two other weights (w_1 and w_2) determine the slope of the plane in the two different dimensions. If the weights are 0, then the plane will be parallel to the x_1 and x_2 axes, while a positive or negative value will cause the plane to tilt. This is shown in the two lower charts

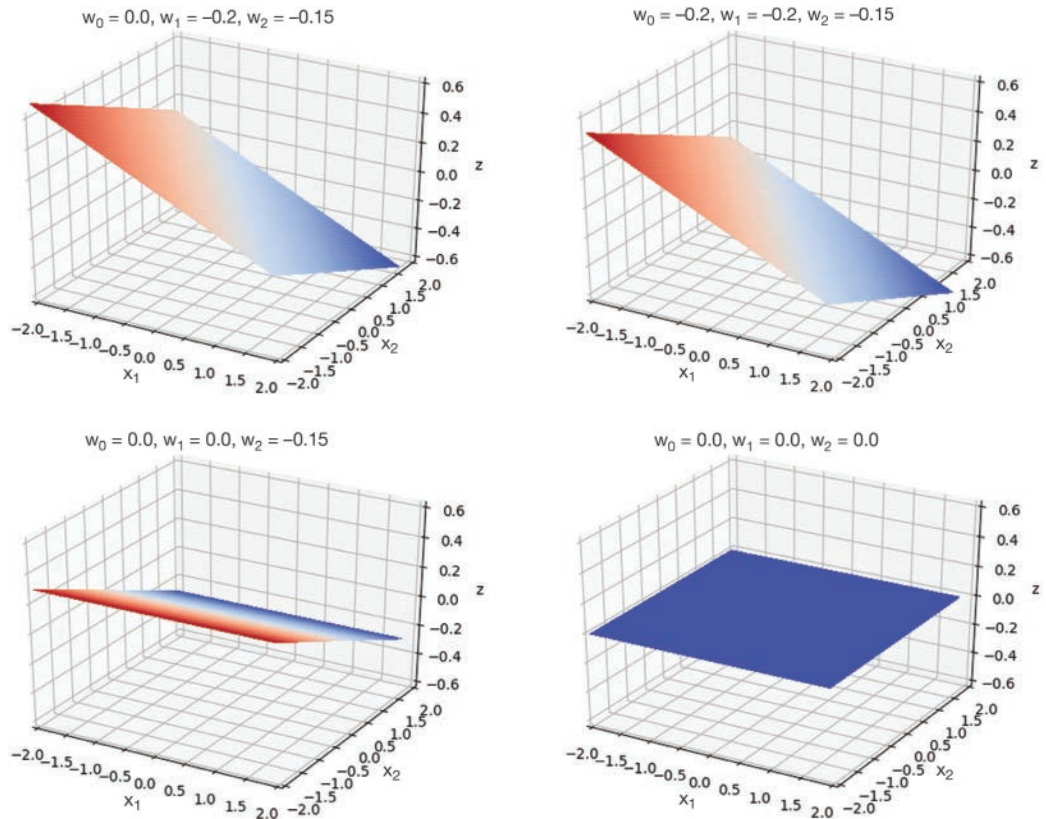


Figure 1-11 Decision surface defined by two-input perceptron. The surface orientation changes as the weights are modified.

in Figure 1-11, where in addition to setting w_0 to 0, we set w_1 to 0 in the lower left chart and all weights to 0 in the lower right chart and end up with a level plane.

This whole discussion has been centered on a perceptron with two inputs (in addition to the bias input) because it makes it possible to visualize the function. Adding more inputs is straightforward, but visualizing it is much harder because humans are inherently bad at visualizing more than three dimensions. The trick is to not even try to visualize but instead try to understand it from a mathematical point of view.

An example of this is that, as previously mentioned, the perceptron has the restriction that it can distinguish between two classes only if they are linearly separable, which for a two-input perceptron (two dimensions) means that they can be separated by a straight line. For a three-input neuron (three dimensions), it means that they can be separated by a plane. We do not delve into details here, but for readers who are already familiar with linear algebra, we point out that for the general case of a perceptron with n inputs (n dimensions), two classes are linearly separable if they can be separated by an $n-1$ dimensional hyperplane. This is a fairly abstract discussion, but do not worry if you have a hard time following it. Knowing about hyperplanes is not required to be able to understand the rest of the book.

Finally, there is also a geometric interpretation of the computation performed by the perceptron. The perceptron computes a dot product between the two vectors $\mathbf{w}$ and $\mathbf{x}$, and we then apply the sign function to the result. Instead of implementing the dot product as a weighted sum, the dot product of the two vectors can also be computed as

$$\mathbf{w} \cdot \mathbf{x} = |\mathbf{w}| |\mathbf{x}| \cos(\theta)$$

where θ is the angle between the vectors $\mathbf{w}$ and $\mathbf{x}$. Since both $|\mathbf{w}|$ and $|\mathbf{x}|$ are positive, the angle between the two vectors will determine the output of the perceptron (an angle greater than 90 degrees will cause -1 , and an angle less than 90 degrees will cause $+1$ as output).

It turns out that we can use the geometric definition of the dot product to reason about what weights will maximize the weighted sum for a given input, but you can safely ignore this for now if it is the first time you have seen this definition of the dot product.

Understanding the Bias Term

Our description of the perceptron includes the use of a bias term that we did not justify further. In addition, taking a closer look at our equations for the perceptron, a couple of things stick out:

$$y = f(z), \text{ where}$$

$$z = \sum_{i=0}^n w_i x_i$$

$$f(z) = \begin{cases} -1, & z < 0 \\ 1, & z \geq 0 \end{cases}$$

$$x_0 = 1 \text{ (bias term)}$$

In particular, the output values of -1 and 1 and the threshold of 0 seem like they have been chosen somewhat arbitrarily. If you are familiar with digital electronics, it might feel more natural if the output values were 0 and 1 , and you might also feel that a threshold of 0.5 is more appropriate. We will get back to that in Chapter 3, but for now, we focus on the threshold, which is often denoted by θ (Greek letter theta). We could make our perceptron more general by replacing the activation function with the following, where the threshold is simply a parameter θ :

$$f(z) = \begin{cases} -1, & z < \theta \\ 1, & z \geq \theta \end{cases}$$

Looking more closely at the condition that needs to be fulfilled for the output to take on the value 1 , we have

$$z \geq \theta$$

This can be rewritten as

$$z - \theta \geq 0$$

That is, as long as we subtract the threshold from z , we can keep our implementation that uses 0 as its threshold. Looking carefully at our original description of the perceptron, it turns out that we were a little bit sneaky and did this all along by including the bias term x_0 in the sum that computes z . That is, the rationale for the bias term in the first place was to make the perceptron implement an adjustable threshold value. It might seem like we should have subtracted the bias, but it does not matter because we have the associated

weight w_0 , which can be both positive and negative. That is, by adjusting w_0 , our perceptron can be made to implement any arbitrary threshold θ . To be crystal clear, if we want the perceptron to use a threshold of θ , then we set w_0 to be $-\theta$.

Finally, if we ignore the activation function for a moment, we can consider how the bias term affects just the weighted sum z . We saw in the previous section how changing w_0 resulted in the plane sliding up and down along the z -axis. To make it even simpler, consider the lower-dimensional case where we have a straight line instead of a plane. The bias term is simply the intercept term b in the equation for a straight line:

$$y = mx + b$$

What we just described in this section does not change how we use the perceptron. It is just a justification for why we implemented it the way we did in the first place. It is also helpful to know when reading other texts that might use an explicit threshold instead of a bias term.

Concluding Remarks on the Perceptron

In this chapter, we introduced the perceptron and looked at it from a couple of different angles. We showed how it can be used to implement a logical function, starting with NAND. A key reason for starting by using the perceptron to implement logical functions is that it quickly leads to one of the perceptron's limitations, as we saw when trying to implement the XOR function. This then explained the need for connecting multiple perceptrons into a network.

In reality, when working with neural networks and DL, we typically do not think about the perceptron or other neurons in terms of logical gates. A perhaps more common view of looking at the perceptron is as a binary classifier. We feed the perceptron an input example consisting of a vector of input values. The perceptron classifies this input example as belonging to one of two classes. The vector of input values typically contains many more variables than two. For example, in a medical setting, the values in the vector might represent data about a patient, such as age, sex, and various laboratory results. The task for the perceptron is to classify whether the input values indicate that the patient has a specific medical condition. In reality, because of the limitations of the perceptron, this classifier would likely not be very good. Instead, just as for the XOR example, it is likely that

a network of neurons would do better. We will see plenty of examples of such networks in the remainder of this book.

We also introduced the perceptron learning algorithm and showed how it learns a simple task. However, we never described *why* it works. That is the main topic of Chapter 2, which will also serve as a steppingstone for describing the backpropagation algorithm used to train multilevel networks in Chapter 3.

This page intentionally left blank

Chapter 2

Gradient-Based Learning

In this chapter, we describe how the perceptron learning algorithm works, which we then build upon in Chapter 3, “Sigmoid Neurons and Backpropagation,” by extending it to multilevel networks. These two chapters contain more mathematical content than other chapters in this book, but we also describe the concepts in an intuitive manner for readers who do not like reading mathematical formulas.

Intuitive Explanation of the Perceptron Learning Algorithm

In Chapter 1, “The Rosenblatt Perceptron,” we presented and used the perceptron learning algorithm, but we did not explain why it works. Let us now look at what the learning algorithm does. To refresh our memory, the weight adjustment step in the perceptron learning algorithm is first restated in Code Snippet 2-1, where

Code Snippet 2-1 Weight Update Step of Perceptron Learning Algorithm

```
for i in range(len(w)):
    w[i] += (y * LEARNING_RATE * x[i])
```

w is an array representing the weight vector, x is an array representing the input vector, and y is the desired output.

If an example is presented to the perceptron and the perceptron correctly predicts the output, we do not adjust any weights at all (the code that ensures this is not shown in the snippet). This makes sense because if the current weights already result in the correct output, there is no good reason to adjust them.

In the cases where the perceptron predicts the outputs incorrectly, we need to adjust the weights as shown in Code Snippet 2-1, and we see that the weight adjustment is computed by combining the desired y value, the input value, and a parameter known as `LEARNING_RATE`. We now show why the weights are adjusted the way they are. Let us consider three different training examples where x_0 represents the bias input that is always 1:

Training example 1: $x_0 = 1, x_1 = 0, x_2 = 0, y = 1$

Training example 2: $x_0 = 1, x_1 = 0, x_2 = 1.5, y = -1$

Training example 3: $x_0 = 1, x_1 = -1.5, x_2 = 0, y = 1$

We further know that the z -value (the input to the signum function) for our perceptron is computed as

$$z = w_0x_0 + w_1x_1 + w_2x_2$$

For training example 1, the result is

$$z = w_0 \cdot 1 + w_1 \cdot 0 + w_2 \cdot 0 = w_0$$

Clearly, w_1 and w_2 do not affect the result, so the only weight that makes sense to adjust is w_0 . Further, if the desired output value is positive ($y = 1$), then we would want to increase the value of w_0 . On the other hand, if the desired output value is negative ($y = -1$), then we want to decrease the value of w_0 . Assuming that the `LEARNING_RATE` parameter is positive, Code Snippet 2-1 does exactly this when it adjusts w_i by adding a value that is computed as $y * \text{LEARNING_RATE} * x[i]$, where x_1 and x_2 are zero for training example 1 and thus only w_0 will be adjusted.

Doing the same kind of analysis for training example 2, we see that only w_0 and w_2 will be adjusted, both in a negative direction because y is -1 and x_0 and x_2 are positive. Further, the magnitude of the adjustment for w_2 is greater than for w_0 since x_2 is greater than x_1 .

Similarly, for training example 3, only w_0 and w_1 will be adjusted, where w_0 will be adjusted in a positive direction and w_1 will be adjusted in a negative direction because y is positive and x_1 is negative.

To make this even more concrete, we compute the adjustment value for each weight for the three training examples, with an assumed learning rate of 0.1. They are summarized in Table 2-1.

We make a couple of observations:

- The adjustment of the bias weight depends only on the desired output value and will thus be determined by whether the majority of the training examples have positive or negative desired outputs.¹
- For a given training example, only the weights that can significantly affect the output will see a significant adjustment, because the adjustments are proportional to the input values. In the extreme, where an input value is 0 for a training example, its corresponding weight will see zero adjustment.

This makes much sense. In a case where more than 50% of the training examples have the same output value, adjusting the bias weight toward that output value will make the perceptron be right more than 50% of the time if all other weights are 0. It also makes sense to not adjust weights that do not have a big impact on a given training example, which will likely do more harm than good because the weight could have a big impact on other training examples.

In Chapter 1, we described how the z -value of a two-input (plus bias term) perceptron creates a plane in a 3D space (where x_1 is one dimension, x_2 the second, and the resulting value z is the third). One way to visualize the perceptron learning algorithm is to consider how it adjusts the orientation of this plane. Every

Table 2-1 Adjustment Values for Each Weight for the Three Training Examples

	W_0 CHANGE	W_1 CHANGE	W_2 CHANGE
Example 1	$1 \cdot 1 \cdot 0.1 = 0.1$	$1 \cdot 0 \cdot 0.1 = 0$	$1 \cdot 0 \cdot 0.1 = 0$
Example 2	$(-1) \cdot 1 \cdot 0.1 = -0.1$	$(-1) \cdot 0 \cdot 0.1 = 0$	$(-1) \cdot 1.5 \cdot 0.1 = -0.15$
Example 3	$1 \cdot 1 \cdot 0.1 = 0.1$	$1 \cdot (-1.5) \cdot 0.1 = -0.15$	$1 \cdot 0 \cdot 0.1 = 0$

1. Only training examples that are incorrectly predicted will cause an adjustment. Thus, a case with many training examples with positive outputs can still result in a negative bias weight if many of the positive training examples already are correctly predicted and thus do not cause any weight adjustments.

update will adjust the bias weight. This will push the overall plane upward for positive training examples and downward for negative training examples.

For example, close to the z -axis (x_1 and x_2 are small), the bias weight is all that counts. For cases that are further away from the z -axis, the angle of the plane becomes a more significant lever. Thus, for mispredicted learning examples where the x_1 value is big, we make a big change to the weight that determines the tilt angle in the x_1 direction, and the same applies for cases with big x_2 values but in the orthogonal direction. A point on the plane that is located directly on the x_2 axis will not move as we rotate the plane around the x_2 axis, which is what we do when we adjust the weight corresponding to the x_1 value.

An attempt at illustrating this is shown in Figure 2-1, with $w_0 = 1.0$, $w_1 = -1.0$, and $w_2 = -1.0$, which are weights that we could imagine would result from repeatedly applying the weight adjustments from Table 2-1.

Looking at the plane, we can now reason about how it satisfies the three training examples. Because $w_0 = 1.0$, the output will be positive when x_1 and x_2 are close to zero ($z = 1.0$ when x_1 and x_2 are 0), which will ensure that training example 1 is correctly handled. We further see that w_1 is chosen so that the plane is slanted in a direction that z increases as x_1 decreases. This ensures that training example 3 is taken care of because it has a negative x_1 value and wants a positive output. Finally, w_2 is chosen so that the plane is slanted in a direction (around its other axis) that z increases as x_2 decreases. This satisfies training example 2 with its positive x_2 input and desired negative output value.

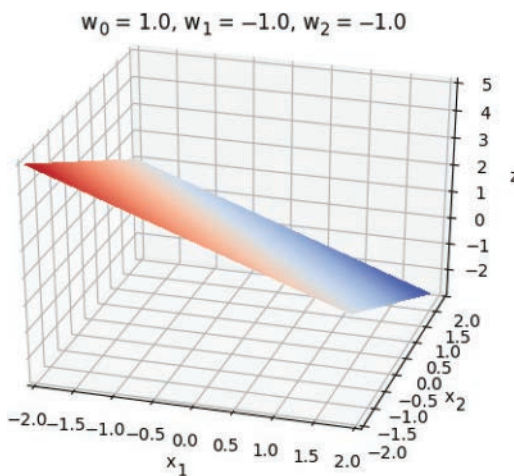


Figure 2-1 Example of weights that correctly predicts all three training examples

We believe that the reasoning is sufficient to give most people an intuitive idea of why the learning algorithm works the way it does. It also turns out that for cases that are linearly separable (i.e., cases where a perceptron has the ability to distinguish between the two classes), this learning algorithm is guaranteed to converge to a solution. This is true regardless of the magnitude of the learning rate parameter. In other words, the value of this parameter will only affect how quickly the algorithm converges.

To prepare ourselves for the learning algorithm for multilevel networks, we would now like to arrive at an analytical explanation of why we adjust the weights the way that we do in the perceptron learning algorithm, but we will first go through some concepts from calculus and numerical optimization that we will build upon.

Derivatives and Optimization Problems

In this section, we briefly introduce the mathematical concepts that we use in this chapter. It is mostly meant as a refresher for readers who have not used calculus lately, so feel free to skip to the next section if that does not apply to you. We start by briefly revisiting what a derivative is. Given a function

$$y = f(x)$$

the derivative of y with respect to x tells us how much the value of y changes given a small change in x . A few common notations are

$$y', f'(x), \frac{dy}{dx}$$

The first notation (y') can be somewhat ambiguous if y is a function of multiple variables, but in this case, where y is only a function of x , the notation is unambiguous. Because our neural networks typically are functions of many variables, we will prefer the two other notations.

Figure 2-2 plots the value of an arbitrary function $y = f(x)$. The plot also illustrates the derivative $f'(x)$ by plotting the tangent line in three different points. The tangent to a curve is a straight line with the same slope (derivative) as the curve at the location where the line touches the curve.

We can make a couple of observations. First, the derivative at the point that minimizes the value of y is 0 (the tangent is horizontal). Second, as we move further away from the minimum, the derivative increases (it decreases if we move in the other direction). We can make use of these observations when solving an

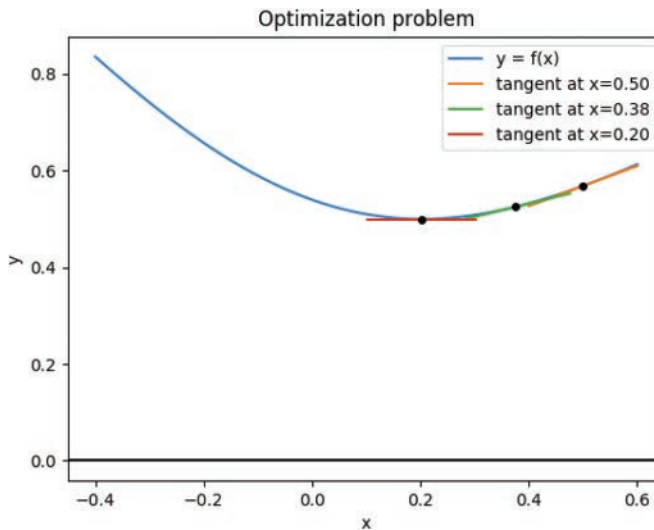


Figure 2-2 Plot showing a curve $y = f(x)$ and its derivative at the minimum value and in two other points

optimization problem in which we want to find what value of the variable x will minimize² the value of the function y . Given an initial value x and its corresponding y , the sign of the derivative indicates in what direction to adjust x to reduce the value of y . Similarly, if we know how to solve x for 0, we will find an extreme point (minimum, maximum, or saddle point)³ of y .

As we saw in Chapter 1, we typically work with many variables. Therefore, before moving on to how to apply these concepts to neural networks, we need to extend them to two or more dimensions. Let us assume that we have a function of two variables, that is, $y = f(x_0, x_1)$, or alternatively, $y = f(\mathbf{x})$, where $\mathbf{x}$ is a 2D vector. This function can be thought of as a landscape that can contain hills and valleys,⁴ as in Figure 2-3.

We can now compute two partial derivatives:

$$\frac{\partial y}{\partial x_0} \text{ and } \frac{\partial y}{\partial x_1}$$

2. We assume that the optimization problem is a minimization problem. There are also maximization problems, but we can convert a maximization problem into a minimization problem by negating the function we want to maximize.

3. We worry only about minima in this book. Further, it is worth noting that these extreme points may well be local extremes. That is, there is no guarantee that a global minimum is found.

4. It can also contain discontinuities, asymptotes, and so on.

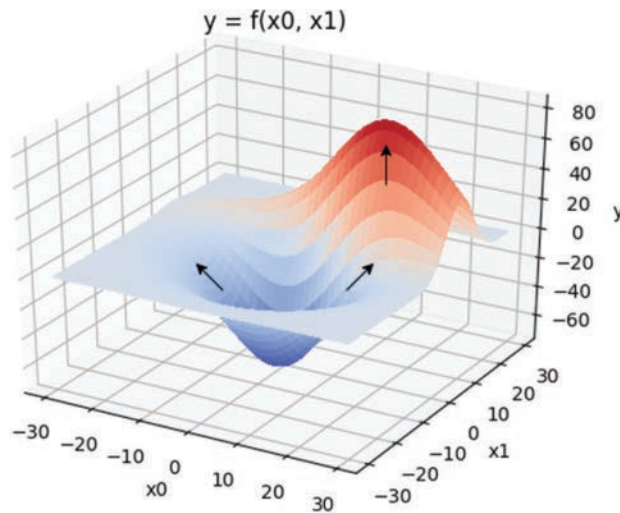


Figure 2-3 Plot of function of two variables and the direction and slope of steepest ascent in three different points

A partial derivative is just like a normal derivative, but we pretend that all but one of the variables are constants. The one variable that we want to compute the derivative with respect to is the only variable that is not treated as a constant. A simple example is if we have the function $y = ax_0 + bx_1$, in which case our two partial derivatives become

$$\frac{\partial y}{\partial x_0} = a$$

$$\frac{\partial y}{\partial x_1} = b$$

If we arrange these partial derivatives in a vector, we get

$$\nabla y = \begin{pmatrix} \frac{\partial y}{\partial x_0} \\ \frac{\partial y}{\partial x_1} \end{pmatrix}$$

which is called the *gradient* of the function—that is, the gradient is a derivative but generalized to a function with multiple variables. The symbol ∇ (upside-down Greek letter delta) is pronounced “nabla.”

The gradient has a geometric interpretation. Being a vector, the gradient consists of a direction and a magnitude.⁵ The direction is defined in the same dimensional space as the inputs to the function. That is, in our example, this is the 2D space represented by the horizontal plane in Figure 2-3. For our example gradient, the direction is (a, b) . Geometrically, this *direction* indicates where to move from a given point (x_0, x_1) in order for the resulting function value (y) to increase the most. That is, it is the direction of the steepest ascent. The *magnitude* of the gradient indicates the slope of the hill in that direction.

The three arrows in Figure 2-3 illustrate both the direction and the slope of the steepest ascent in three point. Each arrow is defined by the gradient in its point, but the arrow does not represent the gradient vector itself. Remember that the direction of the gradient falls in the horizontal plane, whereas the arrows in the figure also have a vertical component that illustrates the slope of the hill in that point.

There is nothing magic about two input dimensions, but we can compute partial derivatives of a function of any number of dimensions and create the gradient by arranging them into a vector. However, this is not possible to visualize in a chart.

Solving a Learning Problem with Gradient Descent

One way to state our learning problem is to identify the weights that, given the input values for a training example, result in the network output matching the desired output for that training example. Mathematically, this is the same as solving the following equation:

$$y - \hat{y} = 0$$

where y is the desired output value and $\hat{y}$ (pronounced “y hat”) is the value predicted by the network. In reality, we do not have just a single training example (data point), but we have a set of training examples that we want our function

5. This explanation assumes that you are familiar with the direction and magnitude of vectors. The magnitude can be computed using the distance formula that is derived from the Pythagorean theorem. Details of this theorem can be found in texts about linear algebra.

to satisfy. We can combine these multiple training examples into a single error metric by computing their mean squared error (MSE):⁶

$$\frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2 \quad (\text{mean squared error})$$

The notation with a superscript number inside parentheses is used to distinguish between different training examples. It is not an indication to raise y to the power of i . Looking closer, it seems like using MSE presents a problem. For most problems, the MSE is strictly greater than 0, so trying to solve it for 0 is impossible. Instead, we will treat our problem as an optimization problem in which we try to find weights that *minimize the value* of the error function.

In most deep learning (DL) problems, it is not feasible to find a closed form solution⁷ to this minimization problem. Instead, a numerical method known as *gradient descent* is used. It is an iterative method in which we start with an initial guess of the solution and then gradually refine it. Gradient descent is illustrated in Figure 2-4, where we start with an initial guess x_0 . We can insert this value into

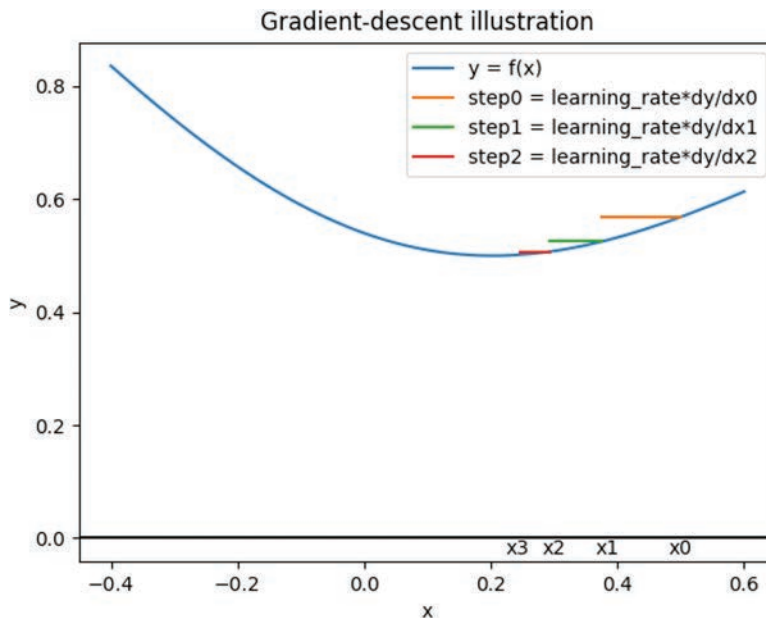


Figure 2-4 Gradient descent in one dimension

6. We will later see that MSE is not necessarily a great error function for some neural networks, but we use it for now because many readers are likely familiar with it.

7. A closed form solution is a solution found by analytically solving an equation to find an exact solution. An alternative is to use a numerical method. A numerical method often results in an approximate solution.

$f(x)$ and compute the corresponding y as well as its derivative. Assuming that we are not already at the minimum value of y , we can now come up with an improved guess x_1 by either increasing or decreasing x_0 slightly. The sign of the derivative indicates whether we should increase or decrease x_0 . A positive slope (as in the figure), indicates that y will decrease if we decrease x . We can then iteratively refine the solution by repeatedly doing small adjustments to x .

Gradient descent is a commonly used learning algorithm in DL.

In addition to indicating in what direction to adjust x , the derivative provides an indication of whether the current value of x is close to or far away from the value that will minimize y . Gradient descent makes use of this property by using the value of the derivative to decide how much to adjust x . This is shown in the update formula used by gradient descent:

$$x_{n+1} = x_n - \eta f'(x_n)$$

where η (Greek letter eta) is a parameter known as the *learning rate*. We see that the step size depends on both the learning rate and the derivative, so the step size will decrease as the derivative decreases. The preceding figure illustrates the behavior of gradient descent using a learning rate (η) of 0.3. We see how the step size decreases as the derivative gets closer to 0. As the algorithm converges at the minimum point, the fact that the derivative approaches 0 implies that the step size also approaches 0.

If the learning rate is set to too large a value, gradient descent can also overshoot the solution and fail to converge. Further, even with a small step size, the algorithm is not guaranteed to find the global minimum because it can get stuck in a local minimum. However, in practice, it has been shown to work well for neural networks.

If you have encountered numerical optimization problems before, chances are that you have used a different iterative algorithm known as the *Newton-Raphson* or *Newton's method*. If you are curious about how it relates to gradient descent, you can find a description in Appendix E.

GRADIENT DESCENT FOR MULTIDIMENSIONAL FUNCTIONS

The preceding example worked with a function of a single variable, but our neural networks are functions of many variables, so we need the ability to

minimize multidimensional functions. Extending it to more dimensions is straightforward. As described in the section “Derivatives and Optimization Problems,” a gradient is a vector consisting of partial derivatives and indicates the direction in the input space that results in the steepest ascent for the function value. Conversely, the negative gradient is the direction of steepest descent, or the direction of the quickest path to reducing the function value. Therefore, if we are at the point $\mathbf{x} = (x_0, x_1)$ and want to minimize y , then we choose our next point as

$$\begin{pmatrix} x_0 \\ x_1 \end{pmatrix} - \eta \nabla y$$

where ∇y is the gradient. This generalizes to functions of any number of dimensions. In other words, if we have a function of n variables, then our gradient will consist of n partial derivatives, and we can compute the next step as

$$\mathbf{x} - \eta \nabla y$$

where both $\mathbf{x}$ and ∇y are vectors consisting of n elements. Figure 2-5 shows gradient descent for a function of two input variables. The function value y gradually decreases as we move from point 1 to point 2 and 3.

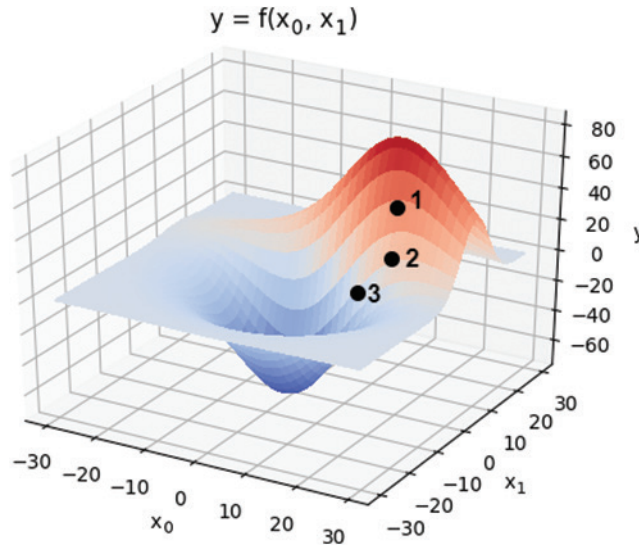


Figure 2-5 Gradient descent for a function of two variables

It is worth noting again that the algorithm can get stuck in a local minimum. There are various ways of trying to avoid this, some of which are mentioned in later chapters but are not discussed in depth in this book.

We are now almost ready to apply gradient descent to our neural networks. First, we need to point out some pitfalls related to working with the multidimensional functions implemented by neural networks.

Constants and Variables in a Network

A key idea when applying gradient descent to our neural network is that we consider input values ($\mathbf{x}$) to be constants, with our goal being to adjust the weights ($\mathbf{w}$), including the bias input weight (w_0). This might seem odd given our description of gradient descent, where we try to find input values that minimize a function. At first sight, for the two-input perceptron, it seems like x_1 and x_2 would be considered input values. That would be true if we had a perceptron with fixed weights and a desired output value and the task at hand was to find the x -values that result in this output value given the fixed weights. However, this is not what we are trying to do with our learning algorithm. The purpose of our learning algorithm is to, given a fixed input (x_1, x_2), adjust the weights (w_0, w_1, w_2) so that the output value takes on the value we want to see. That is, we treat x_1 and x_2 as constants (x_0 as well, but that is always the constant 1, as stated earlier), while we treat w_0, w_1 , and w_2 as variables that we can adjust.

During **learning**, not the inputs ($\mathbf{x}$) but the **weights ($\mathbf{w}$)** are considered to be the **variables** in our function.

To make it more concrete, if we are training a network to distinguish between a dog and a cat, the pixel values would be the inputs ($\mathbf{x}$) to the network. If it turned out that the network incorrectly classified a picture of a dog as being a cat, we would not go ahead and adjust the picture to look more like a cat. Instead, we would adjust the weights of the network to try to make it correctly classify the dog as being a dog.

Analytic Explanation of the Perceptron Learning Algorithm

Now we have the tools needed to explain why the perceptron learning algorithm is defined the way it is. Starting with the two-input perceptron, we have the following variables:

$$\mathbf{w} = \begin{pmatrix} w_0 \\ w_1 \\ w_2 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix}, \quad y$$

The weight vector $\mathbf{w}$ is initialized with arbitrary values. This is our first guess at what the weights should be. We also have a given input combination $\mathbf{x}$ (where x_0 is 1) and its desired output value (y), also known as the ground truth. Let us first consider the case where the current weights result in an output of +1 but the ground truth is -1 . This means that the z -value (the input to the signum function) is positive and we want to drive it down toward (and below) 0. We can do this by applying gradient descent to the following function:⁸

$$z = x_0 w_0 + x_1 w_1 + x_2 w_2$$

where x_0, x_1, x_2 are constants and the weights are treated as variables. First, we need to compute the gradient, which consists of the three partial derivatives with respect to w_0, w_1 , and w_2 . Remember that when computing a partial derivative, all the variables except for the one that we are taking the derivative with respect to are constants, so the gradient simply turns out to be

$$\nabla z = \begin{pmatrix} \frac{\partial z}{\partial w_0} \\ \frac{\partial z}{\partial w_1} \\ \frac{\partial z}{\partial w_2} \end{pmatrix} = \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix}$$

8. In this description of the single perceptron case, we do not formally define an error function that we want to minimize but instead simply identify that we want to reduce the z -value to get the desired output and then use gradient descent to accomplish this. We will use an error function in the next chapter.

Given the current weight vector $\mathbf{w}$, and the gradient ∇z , we can now compute a new attempt at $\mathbf{w}$ that will result in a smaller z -value by using gradient descent. Our new $\mathbf{w}$ will be

$$\mathbf{w} - \eta \nabla z$$

which expands to the following for each component of the vector $\mathbf{w}$:

$$\begin{pmatrix} w_0 - \eta x_0 \\ w_1 - \eta x_1 \\ w_2 - \eta x_2 \end{pmatrix}$$

This is exactly the update rule for the perceptron learning algorithm. That is, the perceptron learning algorithm is equivalent to applying gradient descent to the perceptron function.⁹

If the learning case we considered instead had a ground truth of +1 and our current weight results in -1, then we can multiply all terms by -1 to still make it a minimization problem, and the only difference will be that the gradient will have a different sign, which again makes gradient descent equivalent to the perceptron learning algorithm.

At this point, it is worth pointing out that what we have described so far is an algorithm known as *stochastic gradient descent* (SGD). The distinction between stochastic and true gradient descent is that, with true gradient descent, we would compute the gradient as the mean value of the gradients for all individual training examples, whereas with SGD, we approximate the gradient by computing it for only a single training example. There are also hybrid approaches in which you approximate the gradient by computing a mean of some, but not all, training examples. This approach is studied more in later chapters, but for now we will continue using SGD.

Gradient descent requires you to compute the gradient for **all input examples** before updating the weights, but **stochastic gradient descent** only requires you to compute the gradient for **a single input example**.

We stated the gradient descent algorithm for this problem in vector form. This form applies to any number of dimensions (i.e., it can be used for perceptrons with any number of inputs).

⁹ This statement is not strictly correct: There are some subtleties with respect to the perceptron function not being differentiable in all points, but for the purpose of this discussion, we can ignore that.

Geometric Description of the Perceptron Learning Algorithm

Finally, we offer a geometric explanation to how the perceptron learning algorithm works for readers who think visually. Given that we are limited to three dimensions when plotting, we can only visualize a function with two adjustable parameters. This corresponds to a single-input perceptron, which has w_0 and w_1 as adjustable parameters. Given a specific input example (x_0, x_1) , where $x_0 = 1.0$ as always, our weighted sum z is now a function of the two weights w_0 and w_1 . The independent variables w_0 and w_1 together with the resulting variable z will define a plane oriented in a 3D space. All points on this plane with a positive z -value will result in an output value of $+1$ for the given input values (x_0, x_1) , whereas a negative z -value will result in an output value of -1 for the given input.

Let us assume that with the current input values and weights, the z -value is positive, but the ground truth is negative. The perceptron learning algorithm will simply adjust the weights w_0 and w_1 so that the z -value moves to a different point on this plane, and the point that we move to will be in the direction that the plane is tilted. You can envision that if we place a ball on the point corresponding to (w_0, w_1) and let it roll, it will roll straight toward the point that the perceptron learning algorithm will end up with in the next iteration. This is illustrated in Figure 2-6.

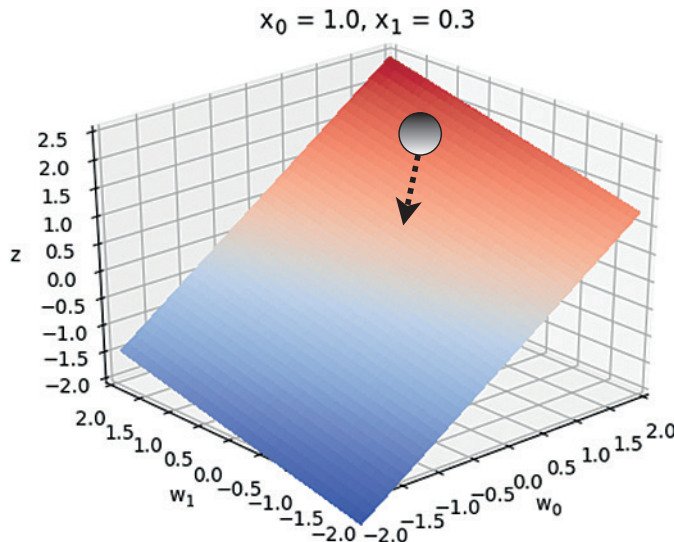


Figure 2-6 Visualization of weight adjustment for a perceptron when desired output is -1

Revisiting Different Types of Perceptron Plots

At this point, we have introduced a number of different plots using two and three dimensions. In some cases, we treated the perceptron inputs ($\mathbf{x}$) as independent variables, and in some cases, we instead switched to making the plot a function of the weights ($\mathbf{w}$). To avoid confusion, we revisit four charts in Figure 2-7 and explain how they relate to each other. The perceptron is presented with an input vector $\mathbf{x}$ and produces a single output y . Internally, it has a weight vector $\mathbf{w}$ and

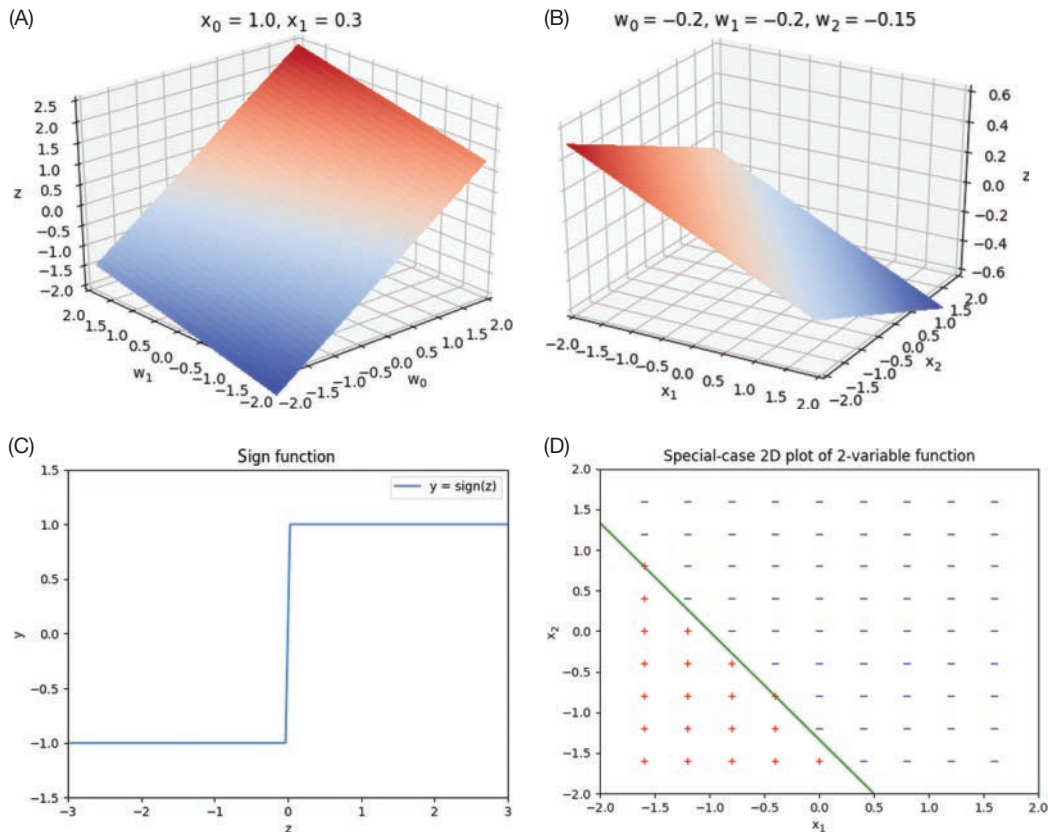


Figure 2-7 (A) Weighted sum z as function of weights w_0, w_1 for one-input perceptron. (B) Weighted sum z as function of inputs x_1, x_2 for two-input perceptron. (C) Perceptron output y as function of weighted sum z . (D) Decision boundary for two-input perceptron acting as a binary classifier.

computes a weighted sum of $\mathbf{x}$ and $\mathbf{w}$. We call the weighted sum z and use it as input to a sign function that produces the output y .

Figure 2-7(A) shows the weighed sum z as a function of two weights w_0 and w_1 . This type of chart is used to understand how the perceptron behavior will change if we adjust its *internal weights* ($\mathbf{w}$). The chart assumes a specific input vector $\mathbf{x}$ ($x_0 = 1.0, x_1 = 0.3$). The weight w_0 is the bias weight, and x_0 is not a true input but is always set to 1.0. That implies that this chart represents a single-input perceptron. We cannot plot this type of chart for perceptrons with two or more inputs.

Figure 2-7(B) shows the weighted sum z as a function of two inputs x_1 and x_2 . This type of chart is used to understand how a perceptron behavior will change when different *input values* ($\mathbf{x}$) are presented to its inputs. The chart assumes a specific set of weights ($w_0 = -0.2, w_1 = -0.2, w_2 = -0.15$). Both x_1 and x_2 are real inputs, so the chart represents a two-input perceptron. That is, when we plot z as a function of the inputs, we can represent a perceptron with more inputs than can be done when we plot z as a function of the weights. This naturally follows from the fact that the bias input x_0 is always 1.0 (treated as a constant), whereas the bias weight w_0 is adjustable (treated as a variable).

Charts A and B visualize the weighted sum z . The two remaining charts visualize the output y as a dependent variable. Figure 2-7(C) simply shows the output y as a function of the weighted sum z . This chart applies to all perceptrons regardless of their weights or number of inputs. As such, the chart might seem somewhat uninteresting, but we see in Chapter 3 how we can replace the sign function by a different function to create a different type of artificial neuron.

Finally, Figure 2-7(D) visualizes the output y as a function of two inputs x_1 and x_2 . At a first glance, this can be confusing when comparing to Figure 2-7(B), which also represents a two-input perceptron. Why must one chart be 3D when another chart can get away with being 2D? The explanation is that the 2D chart exploits the fact that the output has only two possible values (-1 and $+1$), and the regions that take on these two different values are clearly separated. Instead of plotting the output value on its own dimension, we indicate the output value for each point with plus and minus signs. We further draw a line representing the boundary between the two regions. The equation for this line can be derived from the perceptron function. This type of plot is very common when looking at any binary classification problem. The perceptron is just one out of many techniques for binary classification. It belongs to a class of techniques known as *linear classifiers*. Appendix A describes some other linear classifiers and uses the same type of chart to describe their behavior.

This is about as far as we can get in terms of visualizing the behavior of a perceptron. In reality, we often work with many more dimensions, so the attempts at visualizing the process break down. Instead, we will need to trust the mathematics and the formulas we have introduced.

Using a Perceptron to Identify Patterns

Before moving on to extending the learning algorithm to multilevel networks, we will sidetrack a little and look at a different use case for the perceptron. So far, we have studied cases in which the perceptron implements simple two-input logical functions. That is, the perceptron was used to classify data points as belonging to one of two classes, as long as the classes are linearly separable. This is an example of binary classification. An important case for this is to use the perceptron to identify a specific pattern. In such a case, we use the perceptron to classify inputs as either belonging to a specific class of interest or not belonging to that class. That is, we are still doing binary classification, but the other class is “everything else.” Along these lines, we could envision a perceptron that works as a cat identifier. If we present an image of a cat to the perceptron, it will fire, but if we present any other image to the perceptron, it will not fire. If the perceptron does not fire, the only thing we know is that the image was not of a cat, but that does not mean we know what it is. It could be a dog, a boat, a mountain, or anything else. Now, before getting too excited about creating our cat-detecting perceptron, we point out that given the severe limitations of a single perceptron, it is not possible to build a good cat identifier from a single perceptron. We need to combine multiple perceptrons, as we did to solve the XOR problem, and that is something we will do in future chapters. For now, let us consider just a single perceptron and use it to identify some simpler image patterns.

In this example, we analyze a small part of a larger image. We arbitrarily choose to just look at 9 pixels that are arranged in a 3×3 grid. Further, to keep things simple, we assume that a pixel can take on only one of three intensities: white (1.0), gray (0.0), or black (−1.0). This is done to limit the number of training examples. Further, a training example will only consist of combinations of black and white pixels or black and gray pixels. There are no training examples with a combination of gray and white pixels or with black, gray, and white pixels. That is, we have $2^9 = 512$ examples that are black and white and $2^9 = 512$ examples that are black and gray. These two sets overlap at only one place: each contains an image of all-black pixels. So, to be precise, we end up with 1,023 unique training examples.

The task for the perceptron is to signal +1 as output for the specific example that we want it to be able to identify and to signal -1 for all others. To illustrate this, we trained five perceptrons. Each perceptron was trained to identify a specific pattern. The training process consisted of repeatedly presenting all input patterns in random order to the perceptron. We used a ground truth of +1 for the pattern that we wanted the perceptron to learn to identify and -1 for all other examples. Figure 2-8 shows the results.

The five columns correspond to the five different perceptrons. The top row in the figure shows the pattern that we wanted each perceptron to identify. The score under each pattern is the input value to the sign function in the perceptron after it had been trained (i.e., the weighted sum of the inputs when we present the target

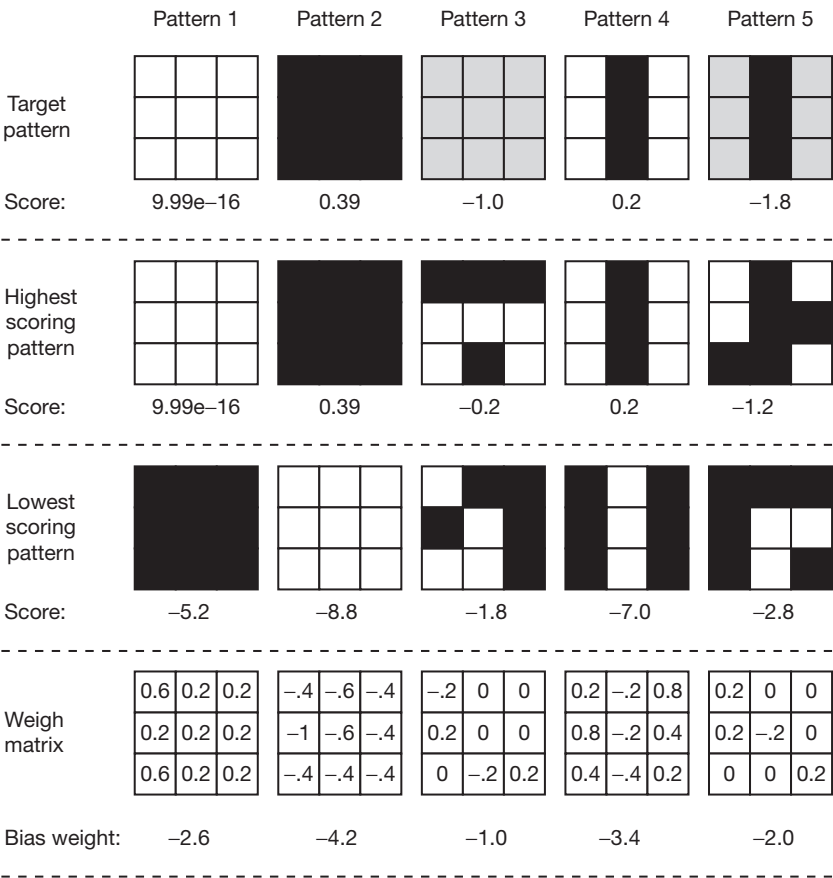


Figure 2-8 Five example patterns, the resulting weights, and highest- and lowest-scoring patterns

pattern to the trained perceptron). The second row shows the highest scoring pattern (we simply presented all combinations to the trained neuron and recorded which patterns resulted in the highest score) and its score. If the highest-scoring pattern is identical to the target pattern, then that means that our classifier is successful at identifying the target pattern. This does not necessarily mean that it does not make any mistakes. For example, a perceptron could output a high score on all patterns, in which case it signals many false positives. The third row shows the lowest-scoring pattern and its score. The bottom row shows the weights after training, including the bias weight.

We can formalize the discussion about **false positives** and introduce the two concepts **precision** and **recall**. This is described in Appendix A.

We make the following observations:

- The perceptron is successful at identifying the black-and-white patterns (1, 2, and 4). Looking at the weights for these cases, they kind of mimic the pattern. Intuitively, this makes much sense if you think about it. If a pixel is white (1.0), you want to multiply it by a positive value to end up with a high score, and if a pixel is black (−1.0), you want to multiply it by a negative value to end up with a high score.
- For the all-white case, the target example barely brings the score above 0. This also makes sense because it implies that other examples that have many, but not all, white pixels will still not be able to get above 0; that is, the number of false positives is limited.
- Even if the input pattern is perfectly symmetric (e.g., all white or all black), the resulting weight pattern is not necessarily symmetric. This is a result of the random initialization of the weights and the algorithm finding one out of many working solutions.
- We see that the perceptron is not successful in perfectly identifying cases in which some of the pixels are gray. In these cases, the algorithm never converged, and we stopped after a fixed number of iterations. Apparently, these are not linearly separable¹⁰ from the other examples. To identify such patterns, we will need to combine multiple neurons as we did for the XOR problem.

10. We have mentioned linear separability a fair number of times, which might give the impression that it is a very important concept to understand. However, the remainder of this book focuses on multilevel networks, which do not suffer from the limitation related to linear separability. Therefore, you do not need to worry much about this concept for the remainder of the book.

- From these examples, it starts getting clear that the perceptron is more powerful than a simple NAND gate. In particular, it can work with real-valued inputs, and we can change its behavior by applying a training algorithm.

In Chapter 1, we briefly described how the dot product can be stated in terms of the angle between the two vectors. For two vectors of a given length, the dot product is maximized when the angle between the two vectors is 0. That is, if the two vectors are of the same length, the dot product is maximized if the two vectors are identical. Therefore, it makes total sense that the weight vector mimics the input pattern.

This concludes our experiment with the perceptron as a pattern (also known as *feature*) identifier, but we use these pattern identifiers as building blocks in future chapters for more advanced image analysis in multilevel networks.

Concluding Remarks on Gradient-Based Learning

This chapter continued to focus on the individual perceptron. We discussed more details of how the perceptron learning algorithm works, from both an intuitive perspective and a more mathematical perspective. As a part of the mathematical description of the perceptron learning algorithm, we introduced how to minimize a function with gradient descent. In addition to describing the learning algorithm, we explored how to use the perceptron as a pattern detector.

It is now time to shift focus from the single perceptron to multilevel networks. The key topic in the next chapter is how to extend the learning algorithm to such networks. The algorithm used to train multilevel networks is based on gradient descent and builds nicely on what you have learned in this chapter.

This page intentionally left blank

Chapter 3

Sigmoid Neurons and Backpropagation

In this chapter, we describe the basic learning algorithm, which virtually all neural-network learning algorithms are variations of. This algorithm is based on a technique known as *backpropagation* (or just *backprop*) and was introduced in the context of neural networks in the mid-1980s. It was a significant step on the path to deep learning (DL). Our impression is that even to many DL practitioners, this algorithm can be a little bit of a mystery because much of it is hidden under the hood of modern DL frameworks. Still, it is crucial to know the basics of how the algorithm works.

At the highest level, the algorithm consists of three simple steps. First, present one or more training examples to the neural network. Second, compare the output of the neural network to the desired value. Finally, adjust the weights to make the output get closer to the desired value. It is as simple as that! It is exactly what we did in the perceptron learning algorithm, and we used gradient descent to determine how much to adjust the weights. For a single perceptron, computing the partial derivatives was trivial. For a multilevel network with multiple neurons per layer, it can be hairy. This is where backprop comes to the rescue. It is a simple and efficient way to compute partial derivatives with respect to weights in a neural network.

Before describing how it works, it is worth pointing out a terminology inconsistency that can be confusing. Our description states that we use *backprop* to compute the partial derivatives that are needed by gradient descent to train the network. An alternative naming convention is to refer to the overall training algorithm as the *backpropagation algorithm*. Regardless of which terminology we use, the overall process consists of the following passes:

- The forward pass, where we present a learning example to the network and compare the network output to the desired value (the ground truth).
- The backward pass, where we compute the partial derivatives with respect to the weights. These derivatives are then used to adjust the weights to make the network output get closer to the ground truth.

The **backpropagation** algorithm consists of a **forward pass** in which training examples are presented to the network. It is followed by a **backward pass** in which weights are adjusted using **gradient descent**. The gradient is computed using the **backpropagation** algorithm.

Throughout this chapter, we build up the description of how the learning algorithm for a multilevel network works, and in the final section, we present a code example for implementing it to solve the XOR problem.

Modified Neurons to Enable Gradient Descent for Multilevel Networks

When we applied gradient descent to the perceptron, we sort of ignored the activation function, that is, the sign function that is applied to the z -value to arrive at the y -value. We did this by using gradient descent to drive z in the desired direction, which we knew would implicitly affect y . That trick cannot be used when working with a multilayer network where the outputs from the activation functions in one layer are used as inputs to the next layer. This is one reason that it was nontrivial to extend the perceptron learning algorithm to multilayer networks.

A key requirement to be able to apply gradient descent is that the function that it is applied to is differentiable, because we need to compute the gradient. The

sign function does not fulfill that requirement due to its discontinuity at zero. Rumelhart, Hinton, and Williams (1986) addressed this when they presented the backpropagation algorithm for multilevel networks. They replaced the sign function with an S-shaped function. One example of such a function is shown in Figure 3-1, which shows the hyperbolic tangent (tanh) function.

The reason that such a function was chosen is obvious when comparing its shape to the sign function. Its shape mimics the sign function, but it is a continuous function and thereby differentiable everywhere. It seems like it is the best of both worlds because the sign function works well for perceptrons, but differentiability is needed for learning.

Another S-shaped function that is important to DL is the *logistic function*, shown in Figure 3-2. To avoid confusion, we should point out something about terminology. Strictly speaking, both tanh and the logistic function belong to a class of functions known as *sigmoid functions*. In older texts on neural networks, *sigmoid function* is commonly used to refer to either the tanh function or the logistic function. However, nowadays the DL field uses *sigmoid function* to refer only to the logistic function. In this book, we interchangeably use the terms logistic function, sigmoid function, and logistic sigmoid function when referring to said function, and we call out tanh separately. The scales of the axes are different for the charts of the tanh

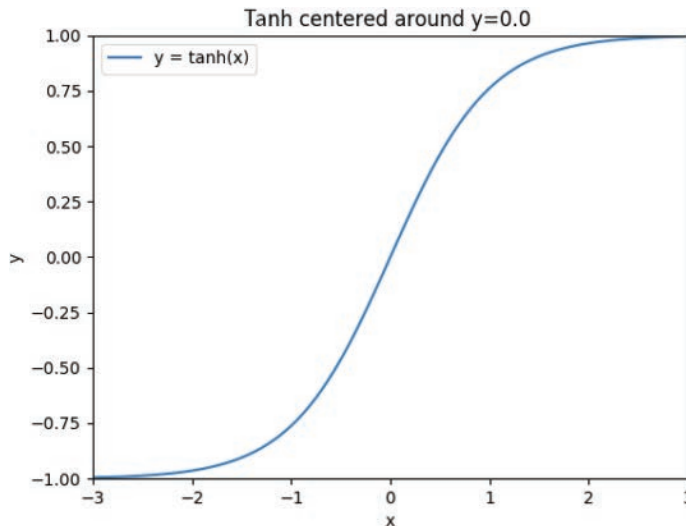


Figure 3-1 Hyperbolic tangent (tanh) function. Note how it is symmetric around 0 on both axes.

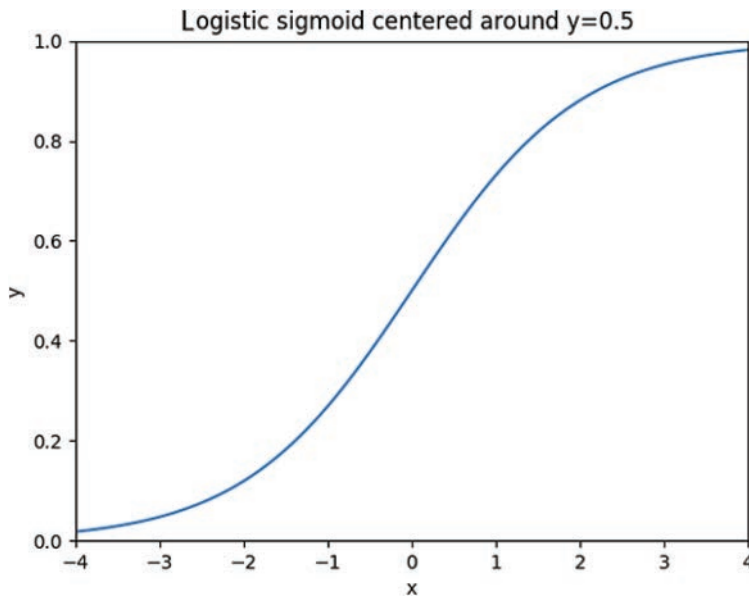


Figure 3-2 Logistic sigmoid function. Note how it is symmetric around 0 on the x-axis and around 0.5 on the y-axis. The scale of both the x-axis and the y-axis is different than in Figure 3-1.

function and the logistic function, so although the curves look similar, they are different.

Before discussing these two functions in more detail, we first introduce their mathematical definitions:

$$\text{Hyperbolic tangent: } \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{e^{2x} - 1}{e^{2x} + 1}$$

$$\text{Logistic sigmoid function: } S(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1}$$

Each function can be stated in a couple of different forms, and you may well run into all of them in other texts on the subject. At a first glance, these definitions might look scary, and it seems magical that anybody could come up with the idea to use these functions when simulating neurons. However, the mathematical definition is less important, and it is the shape that was the driving force. If you study the asymptotes (the output value when x goes to the extreme),

it is fairly easy to convince yourself¹ that as x approaches infinity, the tanh function approaches +1, and as x approaches negative infinity, the tanh function approaches -1, just like the sign function. If we do the same exercise for the logistic sigmoid function, we see that it also approaches +1 as x approaches infinity, whereas it instead approaches 0 as x approaches negative infinity.

We note that both functions are combinations of exponential functions. The shape of an exponential function is similar to a half S as shown in Figure 3-3. If you think about it a little bit, it seems intuitive that it is possible to combine exponential functions to create an S-shape. In particular, an exponential taking x as an argument will dominate for positive x but will be close to 0 for negative x . On the contrary, an exponential taking $-x$ as an argument will exhibit the opposite behavior.

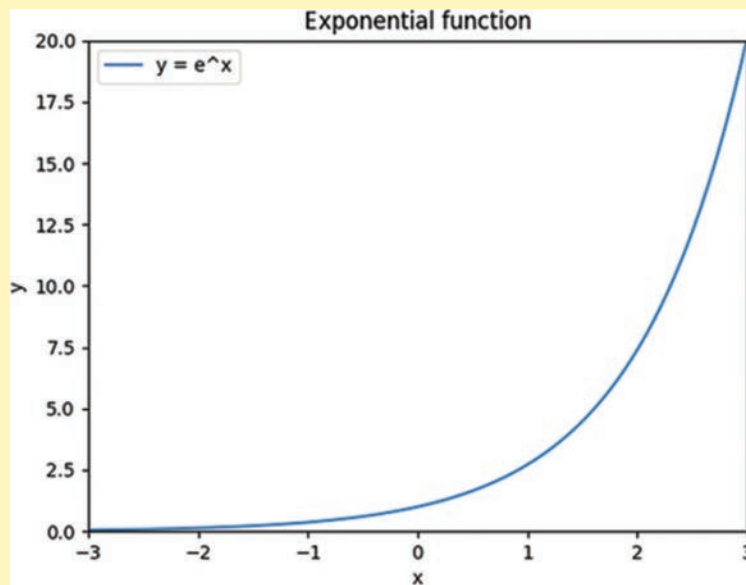


Figure 3-3 Exponential function shaped like the lower half of an S

1. One way of doing this is to play with a spreadsheet and insert different values of x , then look at how the values vary for the various exponential functions and the result of combining them.

We mentioned earlier that the two S-shaped functions are differentiable. Before going into more details of the functions themselves, let us first present their derivatives:

$$\text{Derivative of hyperbolic tangent: } \tanh'(x) = 1 - \tanh^2(x)$$

$$\text{Derivative of logistic sigmoid function: } S'(x) = S(x)(1 - S(x))$$

One key property for both of these functions is that their derivatives can be computed from the function values even if the x -values are not available. Specifically, $\tanh'(x)$ is a function of $\tanh(x)$, and similarly, $S'(x)$ is a function of $S(x)$. To make it more concrete, for $\tanh$, if we have computed $y = \tanh(x)$ for a specific x , then the derivative of the $\tanh$ function for that same x can easily be computed as $1 - y^2$. We will make use of that property later in this chapter.

Let us now look closer at the differences between $\tanh$ and the logistic sigmoid function. Figure 3-4 shows both functions plotted in the same chart. We noted that the $\tanh$ function is more similar to the sign function in that it approaches -1 as x approaches negative infinity, whereas the logistic function bottoms out at 0. We also noted in Chapter 1, “The Rosenblatt Perceptron,” that if you have a background in digital electronics, it might feel more comfortable to use a function with the output range 0 to 1 instead of -1 to $+1$. Another observation is that a range from 0 to 1 can make more sense if we want to interpret the output as a

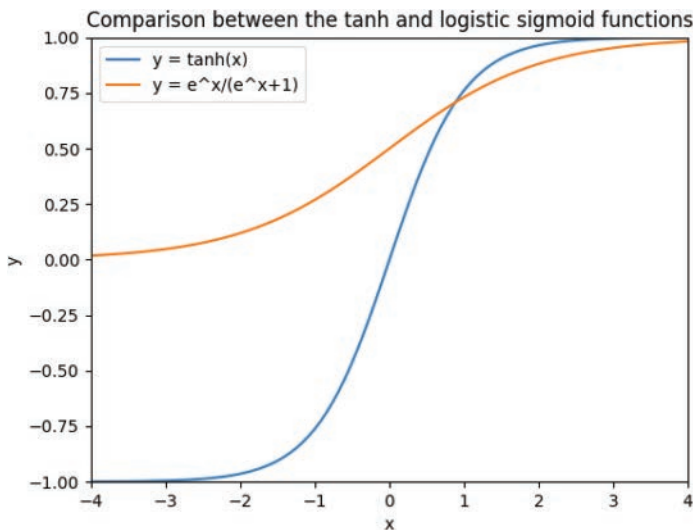


Figure 3-4 Chart comparing the $\tanh$ function and the logistic sigmoid functions

probability—for example, how probable it is that the input to the network is a picture of a cat.

In addition to the output range, there are some subtle differences to consider, namely, the different threshold values (i.e., where the function is centered in the x -direction). Table 3-1 shows three different combinations of threshold value and output range.

From Figure 3-4 and Table 3-1, we can see that both the tanh and logistic sigmoid functions are symmetric around 0 on the x -axis, so their threshold is 0. For the case where the output range is 0 to 1, you may think that a threshold of 0.5 (which we call *symmetric digital*) is more intuitive. To illustrate why, consider what happens if you connect multiple neurons after each other. Further assume that the input to the first neuron is close to its threshold value, and therefore the output of the neuron will fall in the middle of the output range. For both the tanh and symmetric digital neuron, this means that the subsequent neuron will get an input that is close to its threshold value, and again, its output will be in the middle of the output range. On the other hand, in the case of the logistic sigmoid function, the second neuron will get an input that is well above its threshold. Therefore, at a first glance, it seems like a neural net based on neurons using the logistic sigmoid function would be biased toward outputting +1.

The reasoning above ignores two important details. First, the weights can be both positive and negative. Thus, if we have initialized the weights randomly and each neuron is fed by the output from multiple neurons, then even if all the inputs to a neuron in a subsequent layer are close to 0.5, their weighted sum (the input to the activation function) will be close to 0 because about half of the inputs will be multiplied by a negative weight and half will be multiplied by a positive weight. That is, in practice, the symmetric digital function will be biased toward outputting -1 , whereas the logistic sigmoid function would be close to the middle of its output range. Note that tanh works the same either way. The second

Table 3-1 Three Alternative Activation Functions

	SIGN/TANH	LOGISTIC SIGMOID	SYMMETRIC DIGITAL
Max output	1	1	1
Min output	-1	0	0
Threshold	0	0	0.5

detail to consider is the bias input. Let us assume a case where all the weights are positive. Even in this case, it is possible to set the bias weight for the logistic sigmoid function to a value at which the output from the previous layer, combined with the bias, will result in an input to the activation function that is close to the threshold value. As described in Chapter 1, changing the bias term is equivalent to picking a threshold. This means that as long as the bias weight is initialized (or learned) properly, this entire threshold discussion is moot. Neither of the neurons has a fixed threshold, but the threshold can be adjusted by changing the bias weight.

Which Activation Function Should We Use?

This discussion has taken for granted that the activation function should be fairly similar to the sign function but should be differentiable. This is all influenced by history and the all-or-nothing activation (either the neuron fires or it does not fire) described by both McCulloch-Pitts (1943) and Rosenblatt (1958). Rosenblatt mentioned that there were other ideas as well, and it turns out that there are other powerful activation functions that are much different from the sign function. Some of them are not even differentiable in all points even though that was assumed to be a strict requirement in 1986 when the learning algorithm for multilevel networks was introduced. Various examples of these more modern activation functions are introduced in Chapter 5, “Toward DL: Frameworks and Network Tweaks,” and Chapter 6, “Fully Connected Networks Applied to Regression,” but for now we limit ourselves to the logistic sigmoid function and tanh, and an obvious question, then, is which one to pick. As you will see later in this book, this is just one of many similar questions, and there are multiple alternative implementations to choose among. In general, there are no right or wrong answers, but the solution is to experiment and pick whatever method is best for the specific problem that you are working on. However, there are often heuristics that can point you in the right direction as a starting point for these experiments.

When using an S-shaped function as an activation function, we recommend starting with tanh for hidden layers because their output will be centered around 0, which coincides with the threshold of the next layer.² For the output layer, we recommend using the logistic sigmoid function so it can be interpreted as a

2. Assuming that the bias weight is initialized to 0 or randomly with an average of 0.

probability. We will also see that this function works well with a different type of loss function introduced in Chapter 5.

There exist a large number of activation functions where **some, but not all**, are S-shaped functions. Two popular choices are tanh and the logistic sigmoid function. When picking between the two, choose **tanh for hidden layers** and **logistic sigmoid for the output layer**.

The discussion about input and output ranges and their relationships to threshold values focused on the behavior during the forward pass. It turns out that these design choices also influence how easy it is for the backpropagation algorithm to adjust the weights to their desired values. We do not go into details about this, but some further discussions about the difference between the logistic sigmoid function and tanh and how they affect the network training process can be found in a paper by LeCun, Bottou, Orr, and Müller (1998).

Function Composition and the Chain Rule

A central theme in the backpropagation algorithm is to compute the derivative of a *composite function* using the chain rule. In this section, we provide a brief introduction to composite functions and the chain rule. As with the section about partial derivatives and optimization problems in Chapter 2, “Gradient-Based Learning,” this section mainly targets readers who feel rusty with respect to these topics, so feel free to skip to the next section if that does not apply to you.

Function composition is used to combine two or more functions into a new function by using the output value of one function as input value to the next function. Assume that we have two functions:

$$f(x), \text{ and } g(x)$$

Further assume that we use the output of function $g(x)$ as an input to function $f(x)$. Then we can combine them into the composite function

$$h(x) = f(g(x))$$

A common alternative notation is to use the composition operator:

$$h(x) = f \circ g(x), \text{ or just } h = f \circ g$$

The notation with the composition operator is generally preferable when composing multiple functions to avoid all the nested parentheses. In most cases, we use this notation for function composition.

The reason that we bring up function composition is that a multilayer neural network can be written as a composite function. We will see details of that in the next section, but before that, we also need to describe how to compute the derivative for composite functions. The derivative will be needed when applying gradient descent to a multilevel network. The *chain rule* states how we can compute the derivative of a composition of functions. If we have

$$h = f \circ g$$

then the derivative is

$$h' = (f' \circ g)g'$$

Stated differently, if we have

$$z = f(y) \text{ and } y = g(x), \text{ so } z = f \circ g(x)$$

then

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$$

which is also known as *Leibniz's notation*. We will use this notation when applying the chain rule.

In these examples, we only used functions of a single variable. When applying these concepts to neural networks, we will generally work with functions with multiple variables. As an example, assume two input variables for both our functions $g(x_1, x_2)$ and $f(x_3, x_4)$. Further assume that the output of g is used as the second argument to function f . We get the composite function

$$h(x_1, x_2, x_3) = f \circ g = f(x_3, g(x_1, x_2))$$

Just as we did in the Chapter 2, we want to compute partial derivatives of this resulting multivariate function. This is done by treating all other variables as constants, as described in the previous chapter. For the function h that we just described, this implies that we will need to use the chain rule to compute the partial derivatives with respect to variables x_1 and x_2 . When computing the partial derivative with respect to x_3 , g is treated as a constant and we only need to consider the derivative of function f . Detailed examples of this are found in the next section where we do these computations for a neural network.

Using Backpropagation to Compute the Gradient

It is now time to explore how to apply gradient descent to a multilevel network. To keep it simple, we begin with one of the simplest multilevel networks that we can think of, with only a single neuron in each layer. We assume two inputs to the first neuron. The network is shown in Figure 3-5.

We have named the two neurons G and F, where G has three adjustable weights, and F has two adjustable weights. In total, we have a network with five adjustable weights. We want a learning algorithm to automatically find values for these weights that result in the network implementing the desired functionality. The figure also contains something that looks like a neuron to the very right, but this is not a part of the network. Instead, it represents a function needed to determine how right or wrong the network is, which is needed for learning. (This is described in more detail shortly).

We use a weight-naming convention whereby the first letter in the subscript represents the source layer and the second letter represents the destination layer (the layer that the weight is a part of). The digit represents the input number, where 0 is the bias input. That is, w_{xg2} means input number 2 to the g -neuron, which receives its input from layer x . For consistency, we use the same naming

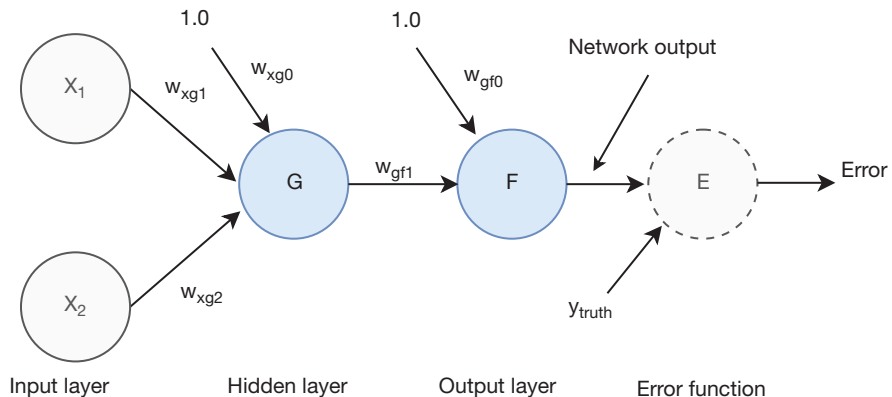


Figure 3-5 Simple two-layer network used to explain backpropagation. The last unit (dashed) is not a part of the network but represents the error function that compares the output to ground truth.

convention for the bias term as for the other terms, although technically it does not have a source layer.

The perceptron studied in Chapter 2 used the sign function as its activation function. As already noted, the sign function is not differentiable in all points. In this chapter, we use activation functions that are differentiable in all points. Neuron G uses $\tanh$ as an activation function, and neuron F uses S (the logistic sigmoid function). This means that a small change to any of the five weights will result in only a small change to the output. When we used a sign function, a small change in a weight would not change anything until the change was big enough to make one perceptron flip, in which case all bets were off because it could easily flip dependent neurons as well.

Taking a step back, our neural network implements the following function:

$$\hat{y} = S(w_{gf0} + w_{gf1} \tanh(w_{xg0} + w_{xg1}x_1 + w_{xg2}x_2))$$

We know of an algorithm (gradient descent) that can be used to minimize a function. To make use of this algorithm we want to define an *error function*, also known as a *loss function*, which has the property that if it is minimized, then the overall network produces the results that we desire. Defining and minimizing error functions is not unique to neural networks but is used in many other contexts as well. One popular error function that comes to mind is the mean squared error (MSE), which we introduced in Chapter 2, and you might already have been familiar with because it is used for linear regression. The way to compute it is to, for each learning example, subtract the predicted value from the ground truth and square this difference, or stated as a mathematical formula:

$$MSE = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

We promised in the preface to not start the book with linear regression, but sometimes we still have to mention it. At least we waited until Chapter 3. For the purpose of this chapter, there is no need to know anything about linear regression, but if you are interested, then you can read more in Appendix A.

Remember that **error function** and **loss function** are two names for the same thing.

There are multiple loss functions to choose among. We use MSE for historical reasons in this section, but in reality, it is not a good choice in combination with a sigmoid activation function.

In other words, MSE is the *mean* (sum divided by m) of the squared error $(y - \hat{y})^2$ for all m training examples. In Chapter 5, we will learn that using MSE as an error function with this type of neural network is not optimal, but we will use it for now just to keep things familiar and simple. That is, assuming a single training example, we want to minimize our loss function $(y - \hat{y})^2$, where $\hat{y}$ was defined above and y is a part of the training example. The following equation combines the formula for MSE with the network function to arrive at the expression of the error function that we want to minimize for a single learning example:

$$Error = \left(y - S(w_{gf0} + w_{gf1} \tanh(w_{xg0} + w_{xg1}x_1 + w_{xg2}x_2)) \right)^2$$

We know that we can minimize this function using gradient descent by computing the gradient of the loss function ($\nabla Error$) with respect to our weights $\mathbf{w}$, then multiplying this gradient by the learning rate (η), and then subtract this result from the initial guess of our weights. That seems straightforward enough, except that computing the gradient of our loss function seems a little bit scary.

A key thing to remember here is that because we want to adjust the weights, we view the **weights $\mathbf{w}$ as variables**, and we view the **inputs $\mathbf{x}$ as constants**; that is, the gradient is computed with respect to $\mathbf{w}$ and not with respect to $\mathbf{x}$.

One brute-force way to solve this problem would be to compute the gradient numerically. We could present an input example to the network and compute and record the output. Then we add Δw to one of the weights and compute the new output and can now compute Δy . An approximation of the partial derivative is now $\Delta y / \Delta w$. Once we have repeated this procedure for all weights, we have computed the gradient. Unfortunately, this is an extremely computationally intensive way of computing our gradient. We need to run through the network $n+1$ times, where n is the number of weights in the network (the $+1$ is necessary because we need the baseline output without any adjustments to the weights).

The backpropagation algorithm solves this problem in an elegant way by computing the gradient analytically in a computationally efficient manner. The